

Monday, August 24, 2026 7:24 AM

1 buy one domain from godaddy

https://account.godaddy.com/products?referrer=sso&auth_reason=1&subdomain=account

Search some domain

GoDaddy | My Products

Ready to add email? Pick the right plan for your growing business. [View Plans →](#)

testlab.com [Search](#) [Set up a free website](#)

What do you want to work on today, Nishant?

123xyz [Search Domains](#)

[RESULTS](#) [HISTORY](#) [FAVORITES](#) [PREMIUM DOMAINS](#)

Domain	Price
PROMOTED 123xyz.diy	₹5,766.27 ₹95.19 for first year
123xyz.site	₹4,099 ₹90 for first year
LOVED BY LOCALS 123xyz.co.in	₹749 ₹1 1st yr only with 3 yr term
123xyz.life	₹5,332.36 ₹380 for first year
123xyztech.xyz	₹2,475.24 ₹189.52 for first year

https://cart.godaddy.com/go/checkout/#/basket

Your Cart

GoDaddy is a trusted growth partner to millions of everyday entrepreneurs.

123xyztech.xyz

THE MORE DOMAIN NAMES YOU OWN, THE MORE PROTECTED YOUR BRAND IS

XYZ Domain Registration

₹189.52

₹2,475.24

92% off

Renews August 2027 for ₹2,475.24

Full Domain Protection at ₹49.92/mo [RECOMMENDED](#)

CHOOSE BY OVER 225,000 CUSTOMERS EACH MONTH

₹599.00/yr

₹999.00/yr

40% off

Professional Email Pro Light

₹29.00/mo

Order Summary (INR)

1 item

Subtotal (INR) ₹189.52

Subtotal does not include applicable taxes and fees

[Have a promo code?](#)

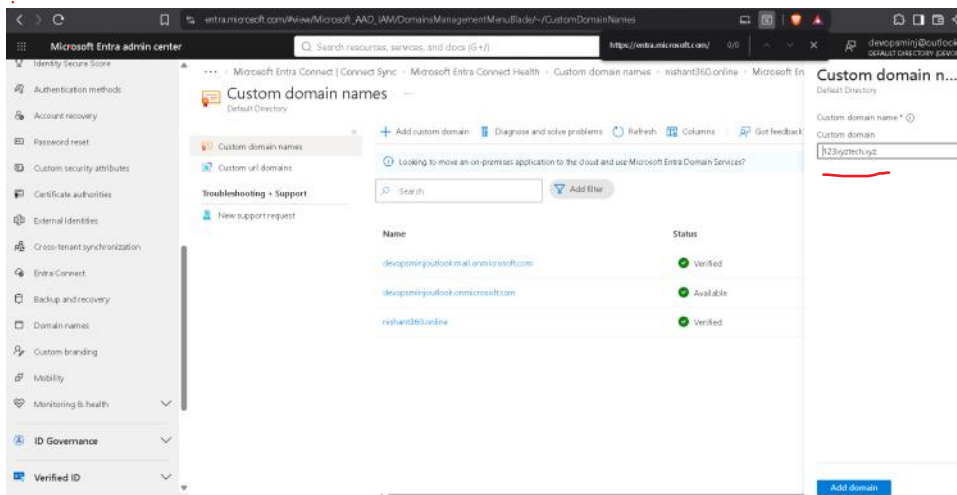
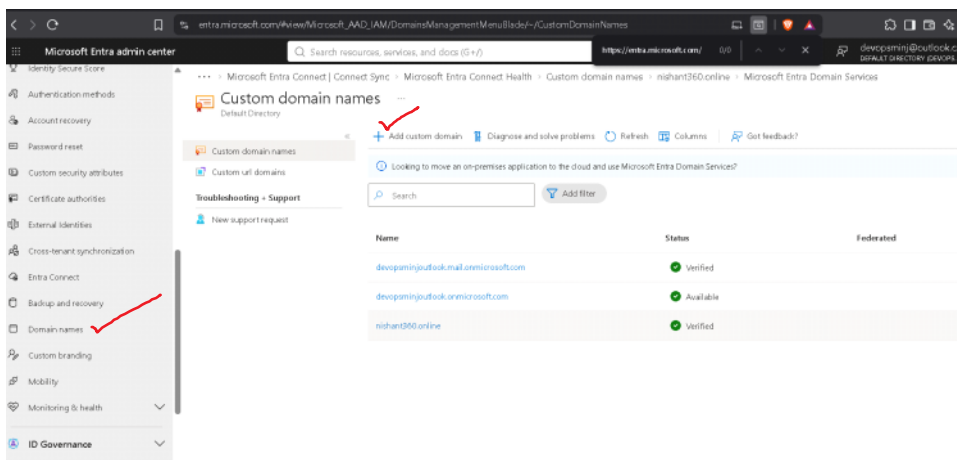
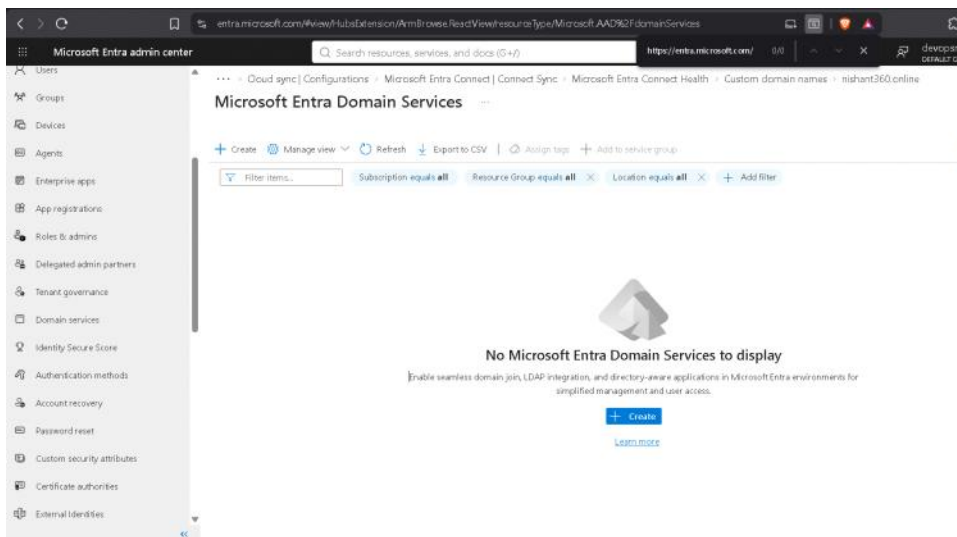
Nice! You saved ₹2,285.72 on your order.

Ready for Checkout

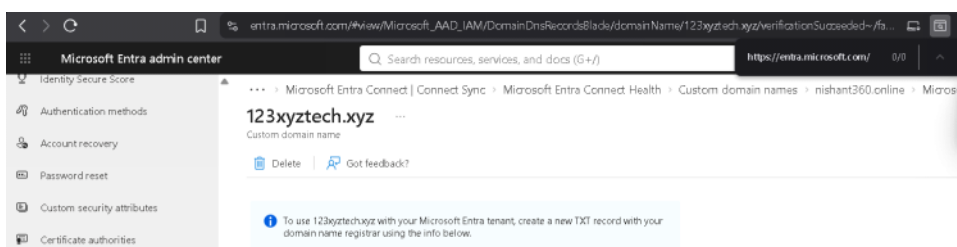
Secure Payment

Then Open the enter admin center to attach the domain

<https://entra.microsoft.com/>



You will get the record like below you need to update them dns record the domain which you own in godaddy



[Delete](#) | [Got feedback?](#)

To use 123egztechxyz with your Microsoft Entra tenant, create a new TXT record with your domain name registrar using the info below.

Record type: **TXT** (selected) | MX

Alias or host name:

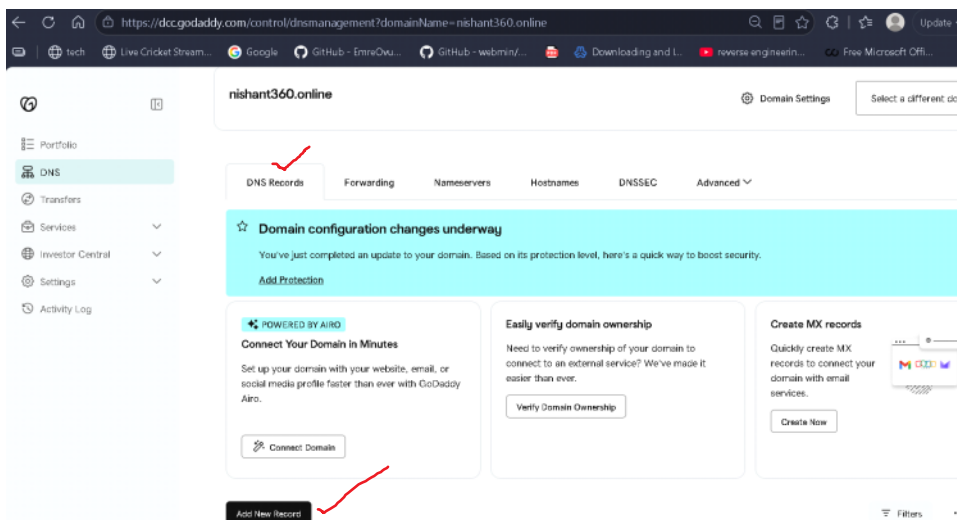
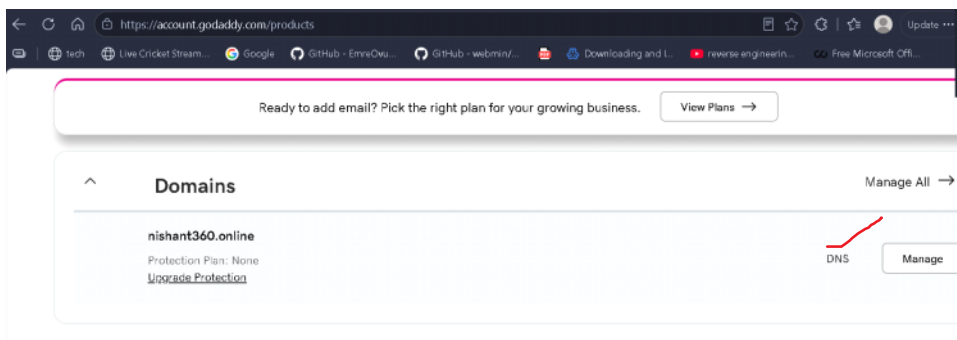
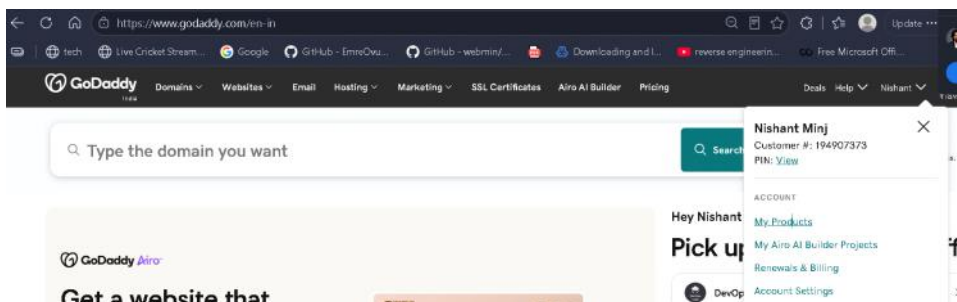
Destination or points to address:

TTL:

[Share these settings via email](#)
 Verification will not succeed until you have configured your domain with your registrar as described above.

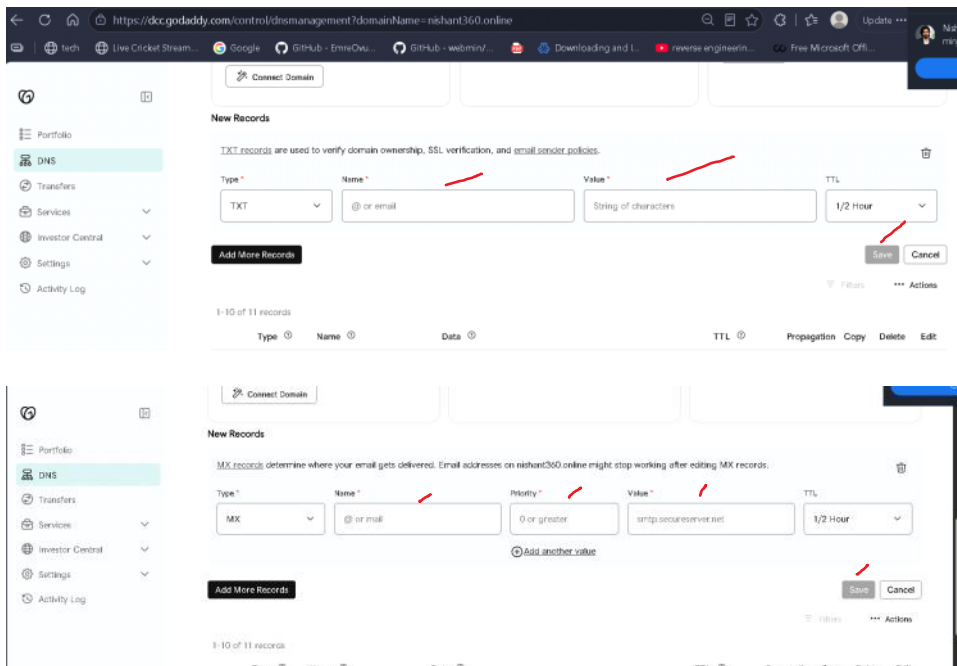
[Verify](#)

Select the product:

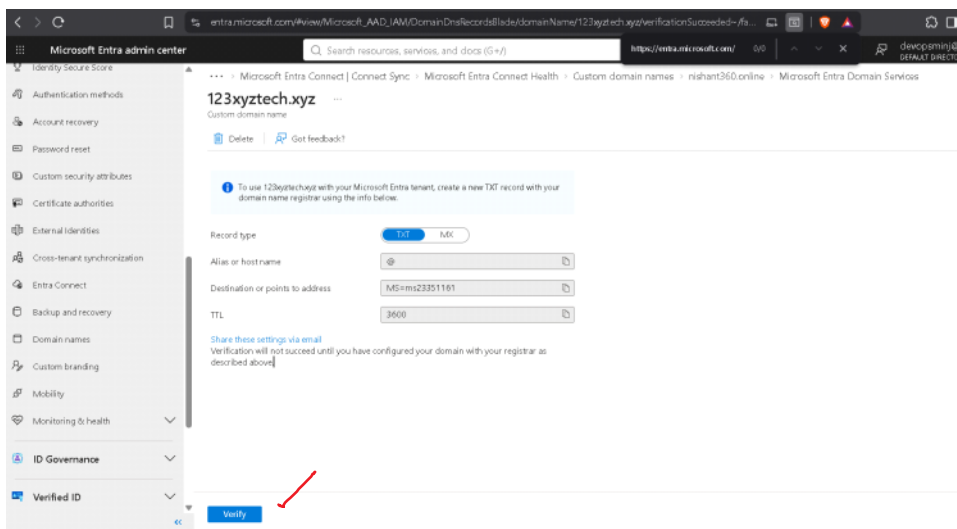


Fill out the details for both txt and mx record :

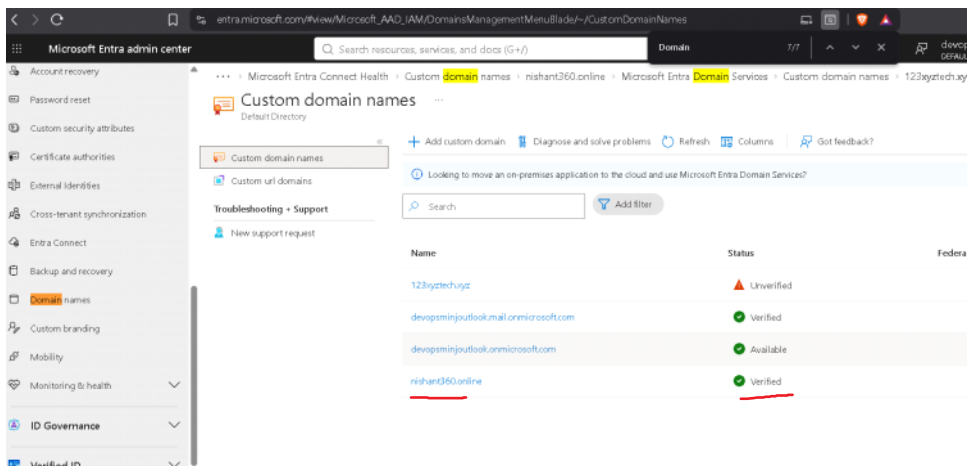




Go the entra admin center then wait for 10-30 minutes and click on verify :

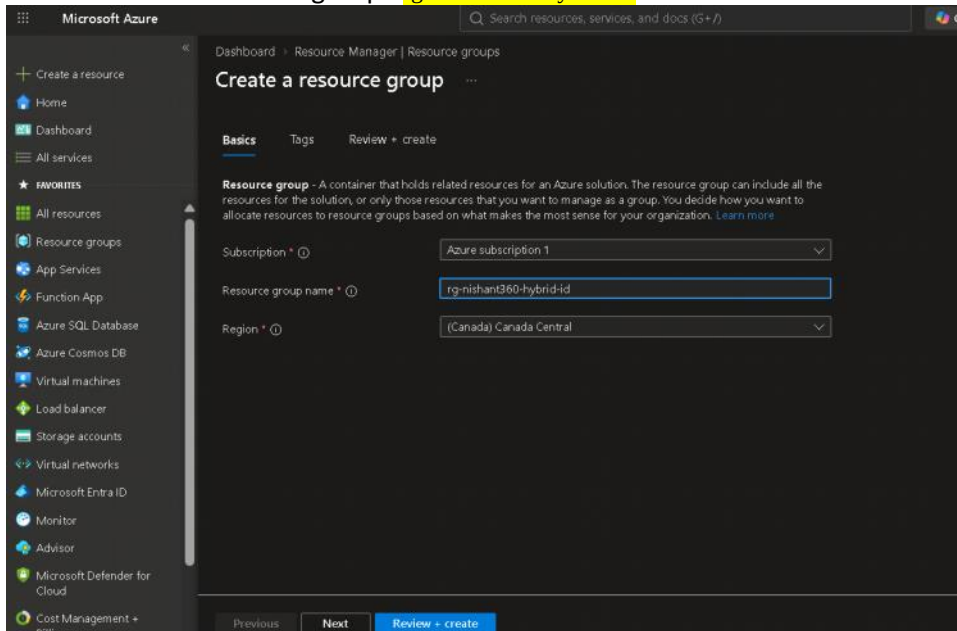


Then final result will be ready like below :

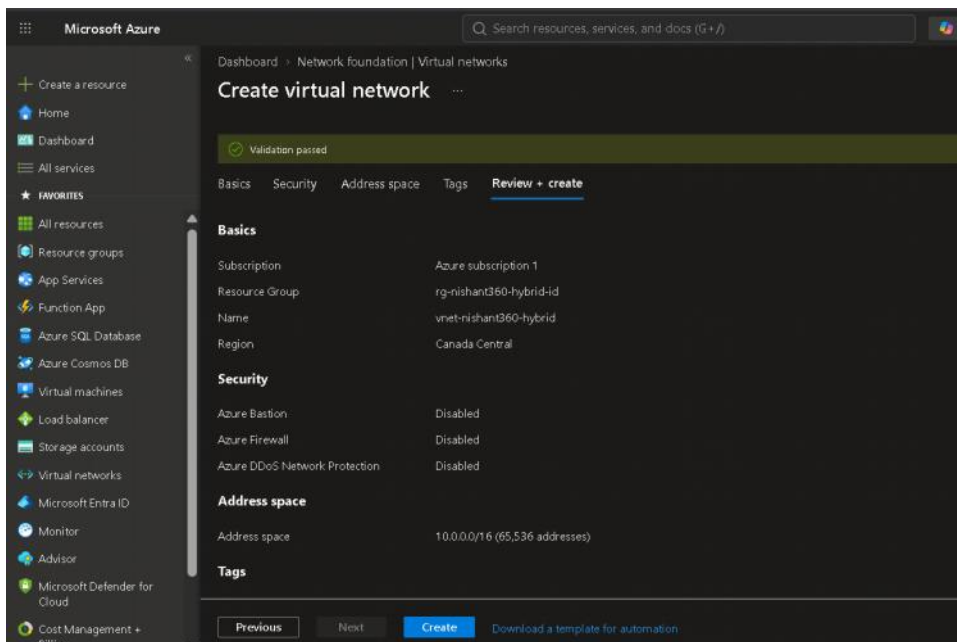


Then create 2 virtual machine with the window 2019 server single vnet and single subnet and NSG must be open for this lab for security reason real time use case we are only opening specific port and protocol, just for the this lab we will open all the protocol and its ports.

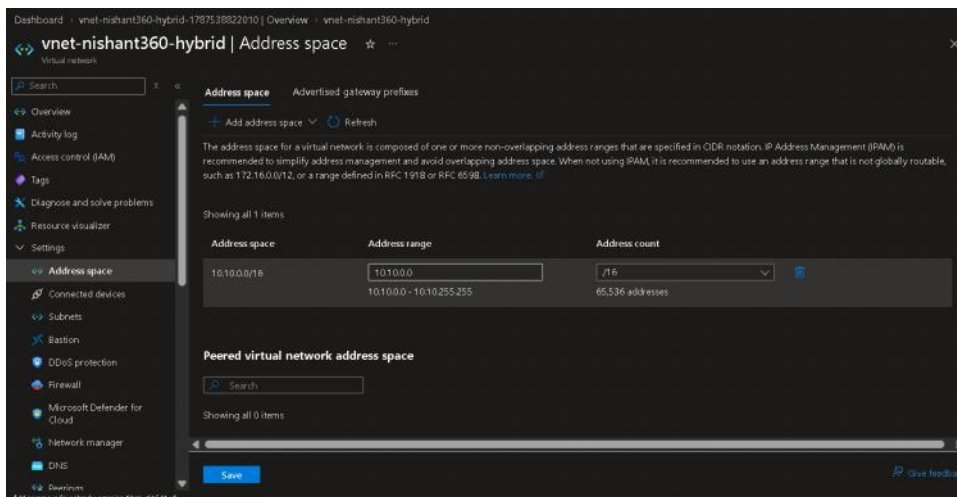
create the first resources group : **rg-nishant360-hybrid-id**



Then create the vnet and subnet use the ip address 10.10.0.0/16 by mistake I used 10.0.0.0/16

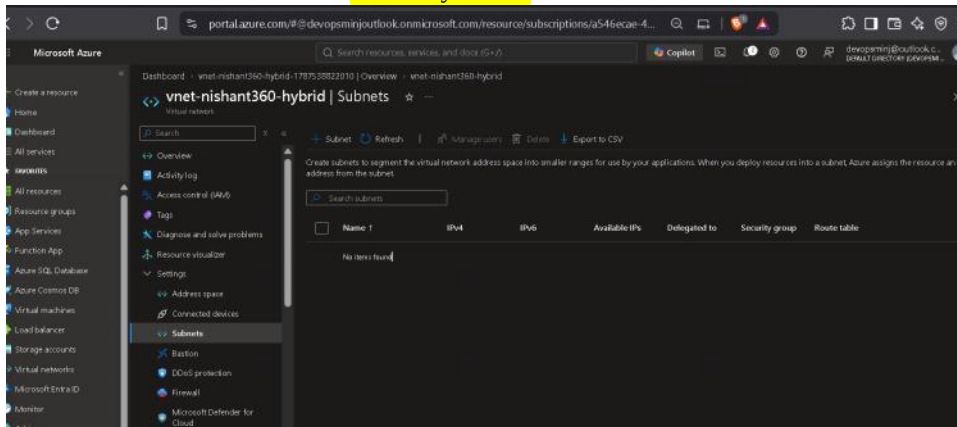


Just correct the ip address cidr as below using the option

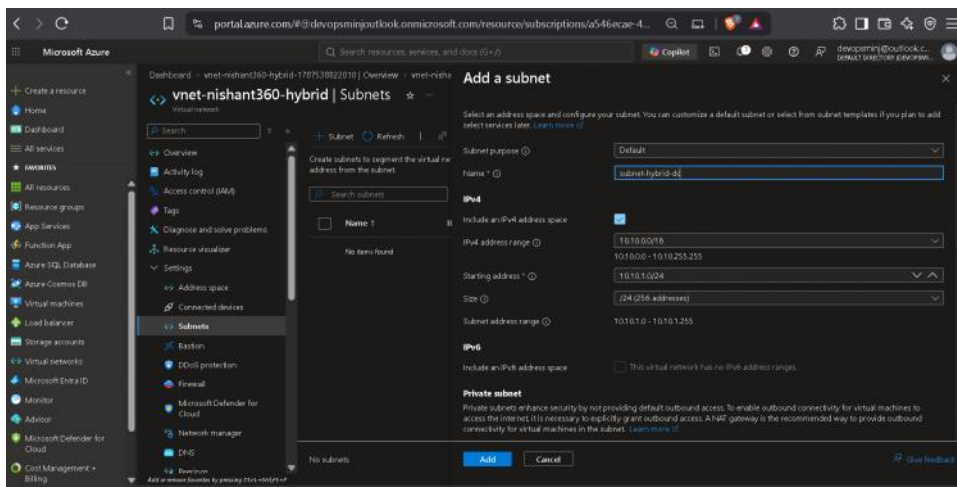


Click on create

Now lets create the subnet : **subnet-hybrid-dc**

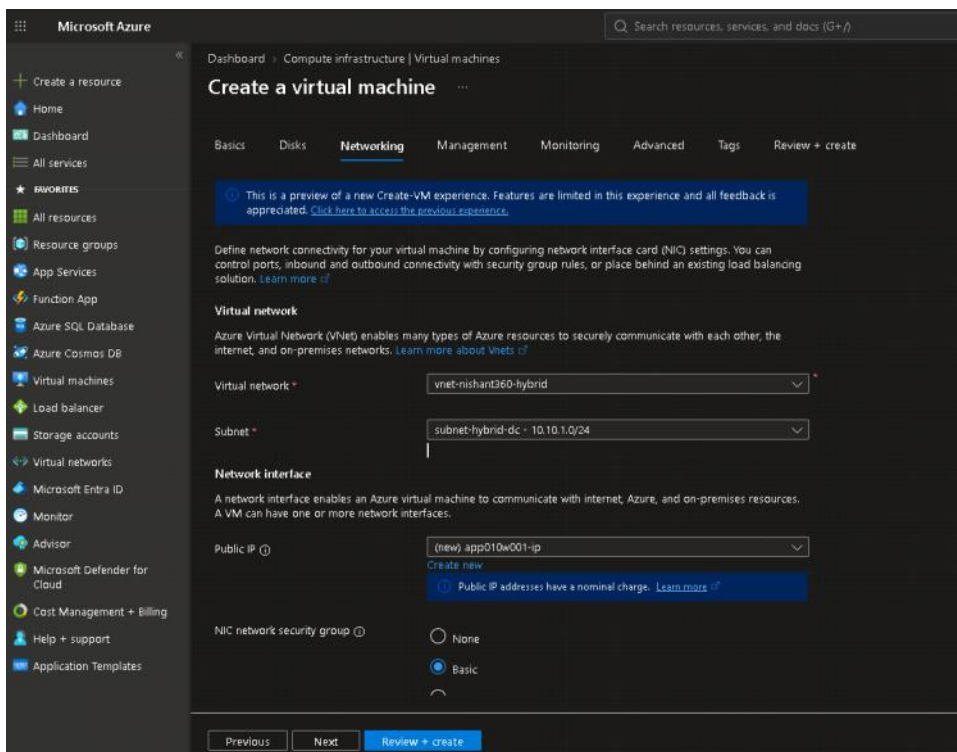
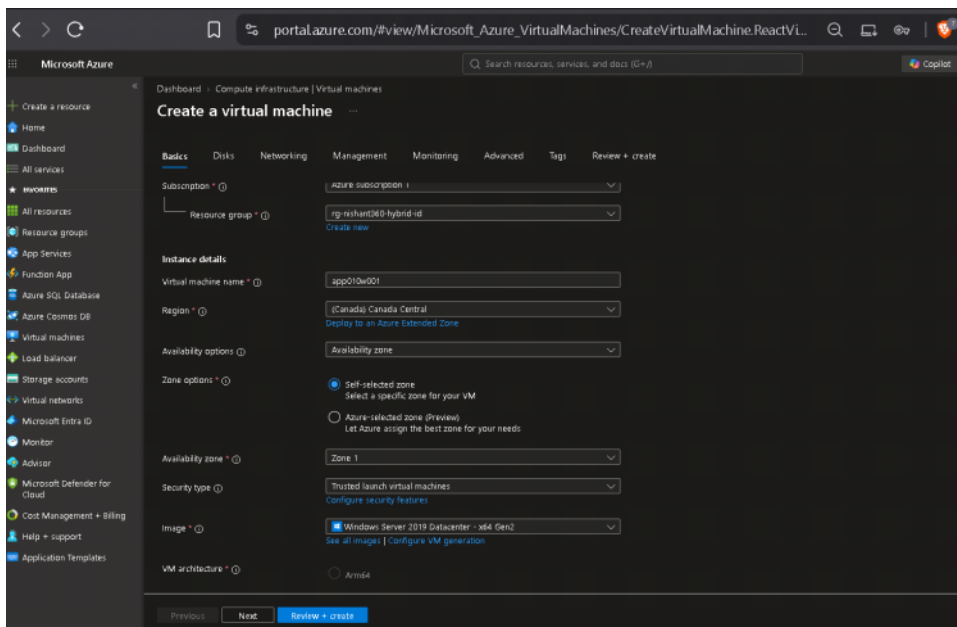
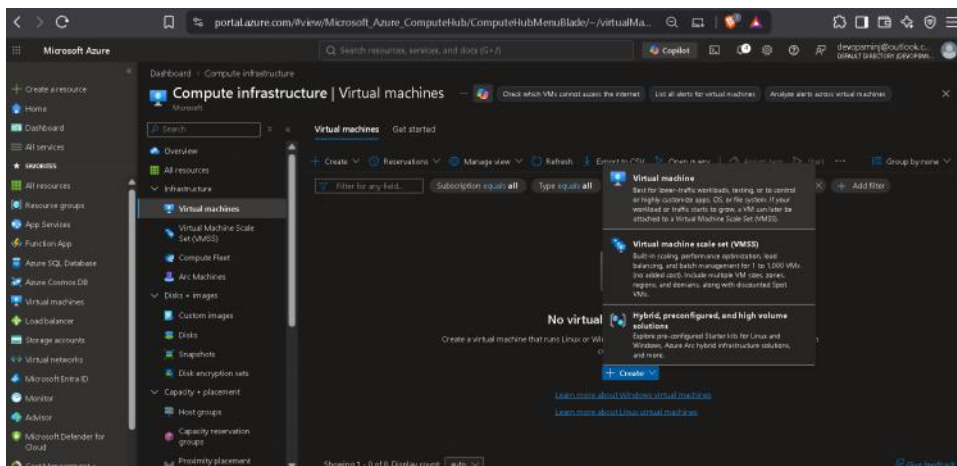


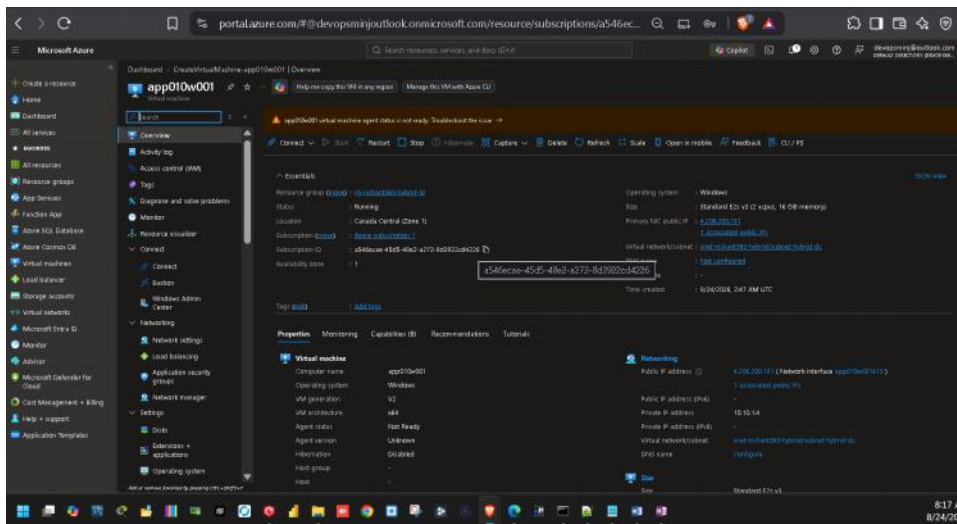
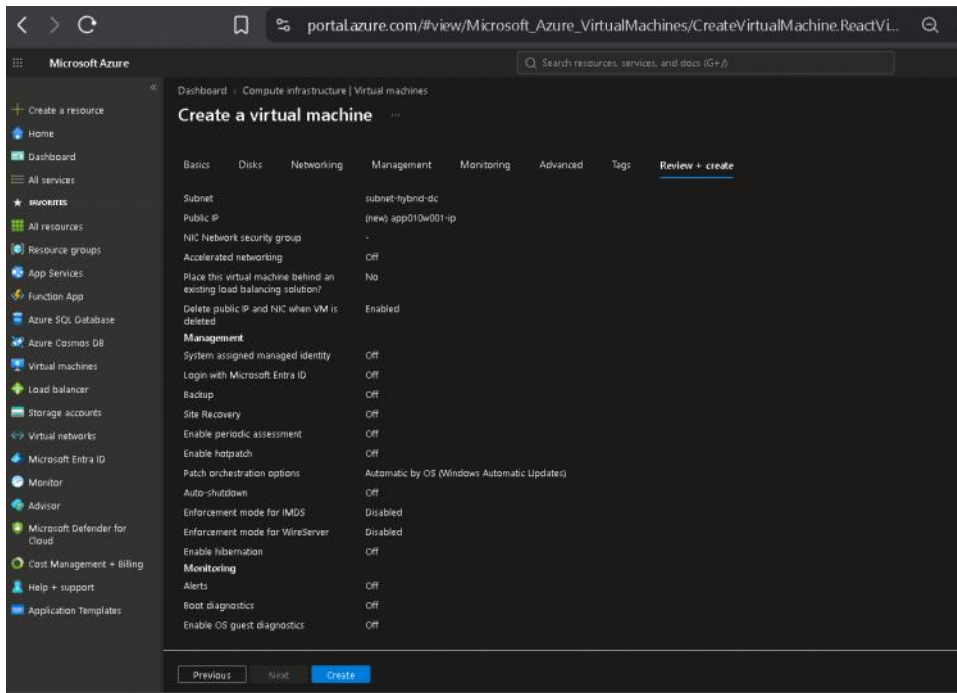
then click on add



Lets create the 2 virtual machine one is for on-prem active directory server and another one is for enter id connect sync agent , what it does is just copy the all the groups to the azure enter id , so entier user will be manage by the active directory.

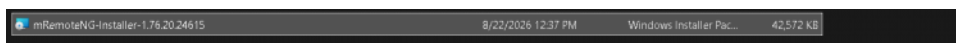
Create Domain Controller ad server app010w001

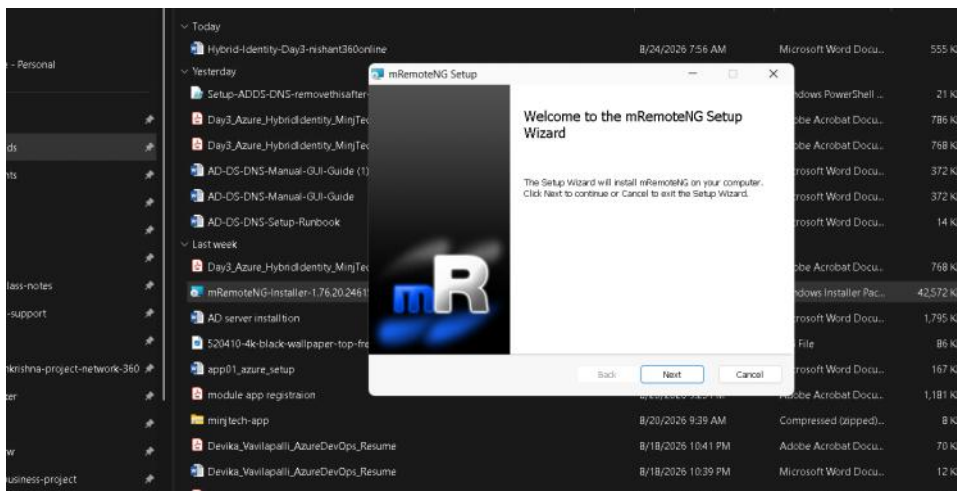
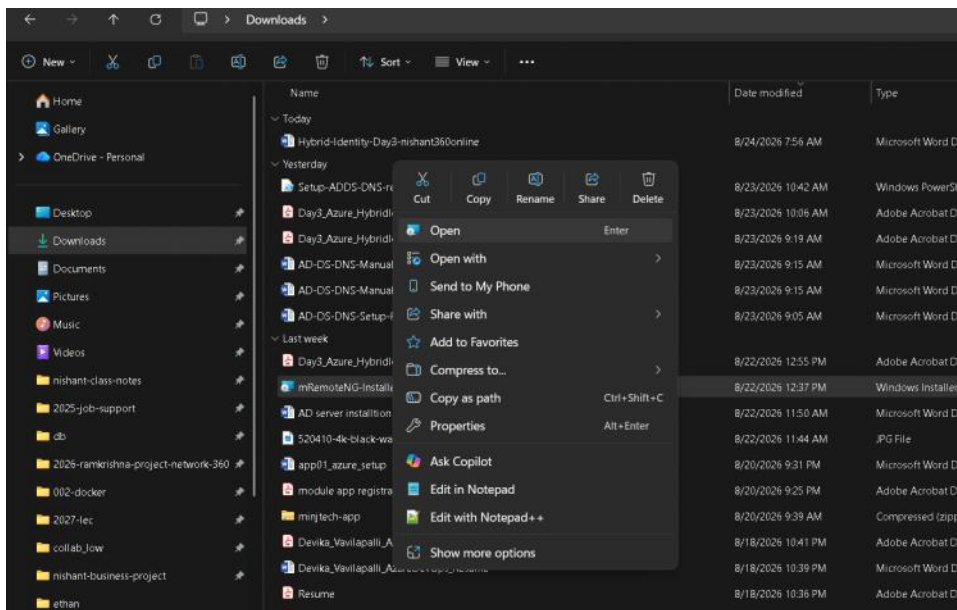


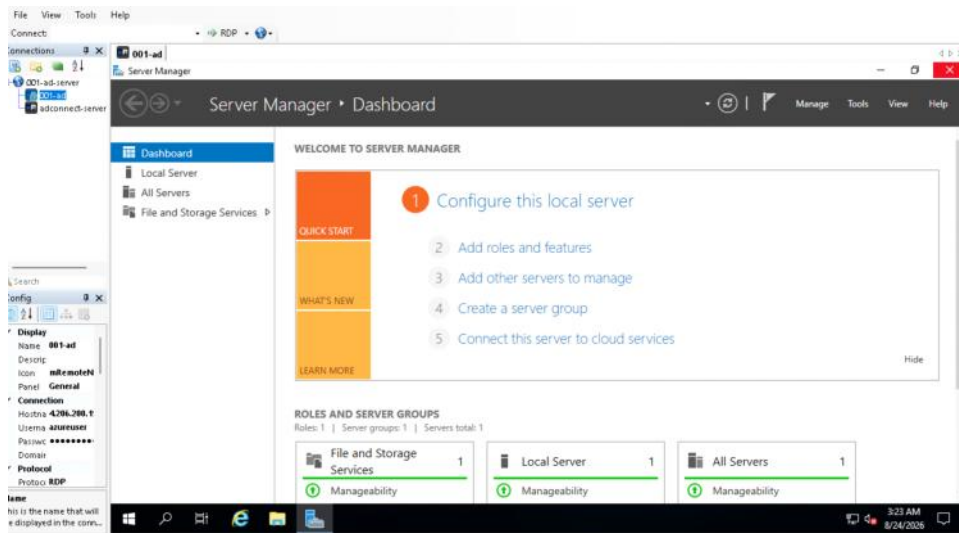


AD server (domain controller is ready) lets do the rdp take the user name
 user: azureuser
 pass: Login%123456
 public ip: use your own vm public ip

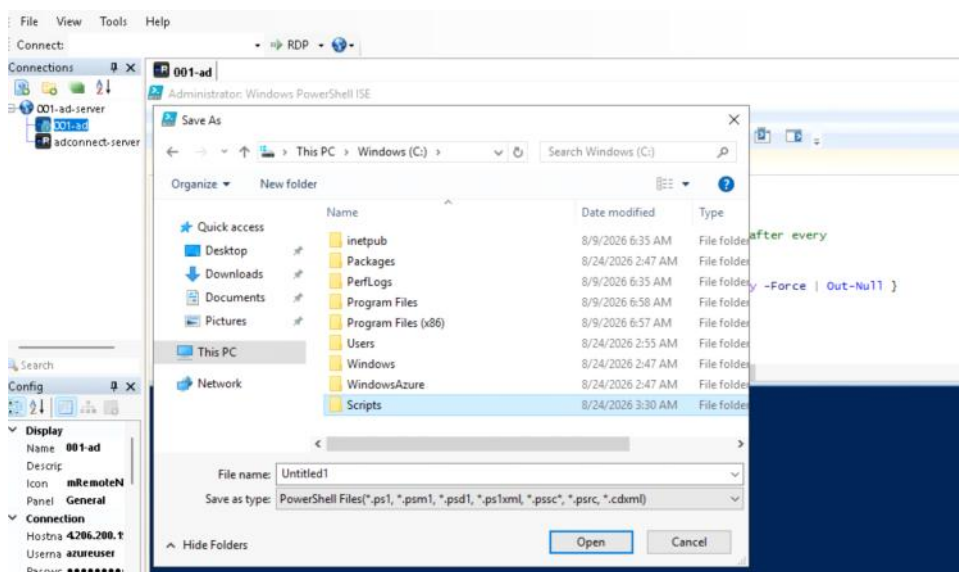
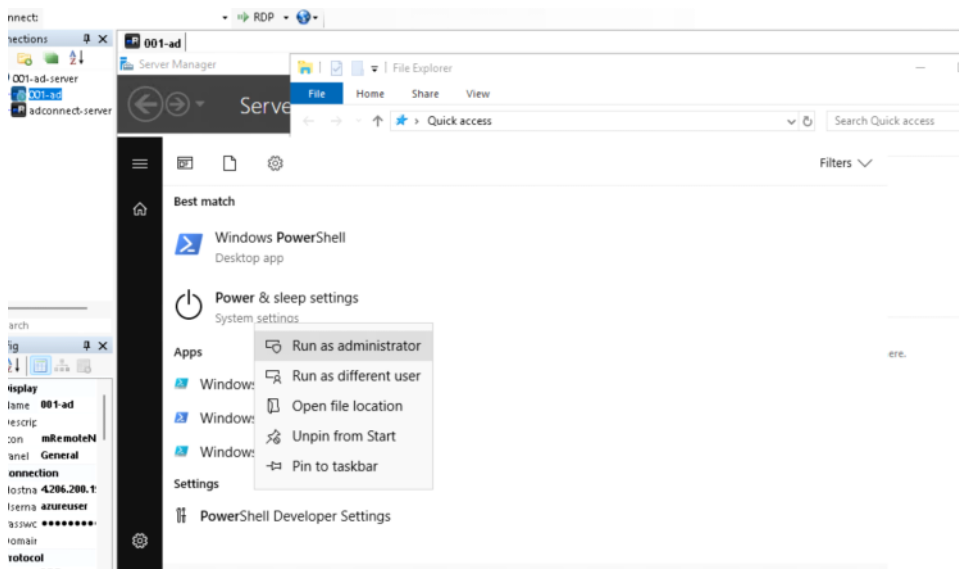
Install below software in your laptop to connect to multiple rdp:

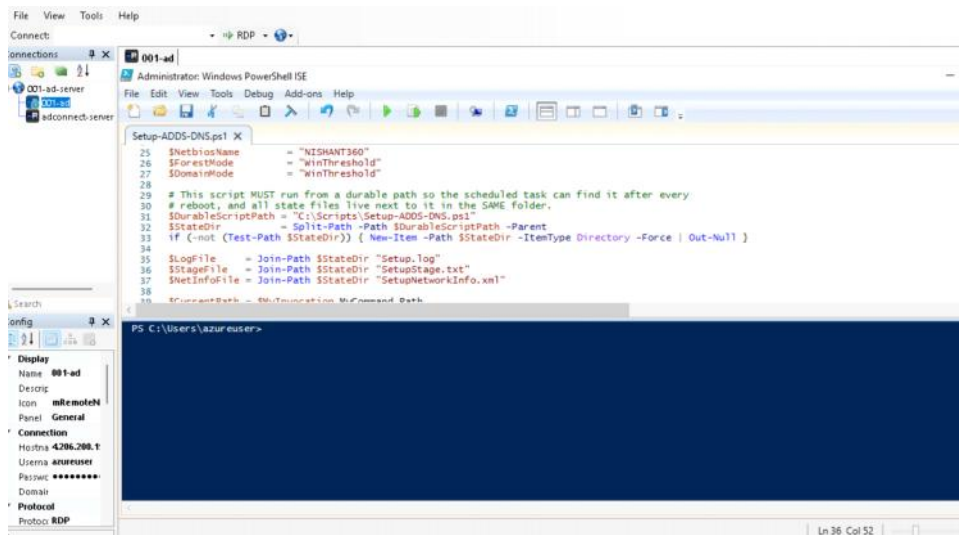






In the c drive create the folder name **Scripts**
and create the script in the location :
 C:\Scripts\Setup-ADDS-DNS.ps1





<https://github.com/minj1992/minj-active-directory/blob/main/Setup-ADDS-DNS-removethisafter-removenishant360.ps1>

```
# =====
# Full unattended setup: rename -> local admin password -> self DNS -> promote
# to AD DS Domain Controller (installs DNS role) -> automatic reverse lookup zone
#
# Designed for Windows Server 2019. Handles the multiple reboots that
# Rename-Computer and Install-ADDSForest require by persisting progress to
# SetupStage.txt (in the same folder as this script) and re-launching itself
# at startup via a scheduled task.
#
# All logging goes to ONE file: Setup.log, in the same folder as this script.
# Every log line is also printed to the console, so you can watch it live.
#
# Run this once, elevated, from a saved copy at C:\Scripts\Setup-ADDS-DNS.ps1:
# PowerShell.exe -NoProfile -ExecutionPolicy Bypass -File C:\Scripts\Setup-ADDS-DNS.ps1
# =====
```

```
# ===== Variables =====
```

```
param ([switch]$RepairReverseDns) # re-run just the reverse-DNS/PTR fix without repeating earlier
stages
```

```
$ErrorActionPreference = "Stop" # make sure real failures land in catch blocks, not get swallowed
```

```
$NewComputerName = "app010w001"
$LocalAdminPassword = "Login%123456" # NOTE: change this / pull from a secret store for real use
$DomainName = "nishant360.online" # replace this domain with your domain
$NetbiosName = "NISHANT360" # keep only you domain name before the dot in caps
$ForestMode = "WinThreshold"
$DomainMode = "WinThreshold"
```

```
# This script MUST run from a durable path so the scheduled task can find it after every
# reboot, and all state files live next to it in the SAME folder.
$DurableScriptPath = "C:\Scripts\Setup-ADDS-DNS.ps1"
$StateDir = Split-Path -Path $DurableScriptPath -Parent
if (-not (Test-Path $StateDir)) { New-Item -Path $StateDir -ItemType Directory -Force | Out-Null }
```

```

$LogFile = Join-Path $StateDir "Setup.log"
$StageFile = Join-Path $StateDir "SetupStage.txt"
$NetInfoFile = Join-Path $StateDir "SetupNetworkInfo.xml"

$CurrentPath = $MyInvocation.MyCommand.Path
if ($CurrentPath -and (Resolve-Path $CurrentPath).Path -ne (Resolve-Path -Path $DurableScriptPath -
ErrorAction SilentlyContinue).Path) {
    Copy-Item -Path $CurrentPath -Destination $DurableScriptPath -Force
} elseif (-not $CurrentPath -or -not (Test-Path $DurableScriptPath)) {
    if (-not (Test-Path $DurableScriptPath)) {
        Write-Warning "Script has no backing file. Save this script as $DurableScriptPath and re-run it from
there so it can survive reboots."
        exit 1
    }
}
$ScriptPath = $DurableScriptPath

# AD DS / DNS Server cmdlets (and several other modules used below) only exist for the
# 64-bit PowerShell host. If we're running as a 32-bit process (e.g. launched from
# "Windows PowerShell ISE (x86)"), relaunch ourselves under the real 64-bit host.
if (-not [Environment]::Is64BitProcess) {
    $sysNative = "$env:windir\Sysnative\WindowsPowerShell\v1.0\powershell.exe"
    $ps64 = if (Test-Path $sysNative) { $sysNative } else { "$env:windir\System32\WindowsPowerShell
\v1.0\powershell.exe" }
    Write-Warning "Running under 32-bit PowerShell; AD DS/DNS cmdlets require 64-bit. Relaunching
under $ps64 ..."
    & $ps64 -NoProfile -ExecutionPolicy Bypass -File $ScriptPath
    exit $LASTEXITCODE
}

# ===== Helpers =====
function Write-Log {
    param ([string]$message)
    $timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
    $line = "$timestamp - $message"
    Write-Host $line
    $line | Out-File -FilePath $LogFile -Append
}

function Test-PendingReboot {
    return (Test-Path 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Component Based
Servicing\RebootPending') -or
        (Test-Path 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\WindowsUpdate\Auto
Update\RebootRequired') -or
        (Test-Path 'HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager
\PendingFileRenameOperations') -or
        ((Get-WmiObject -Class Win32_ComputerSystem).RebootPending)
}

function Register-ContinuationTask {
    # Re-runs this script at every startup (SYSTEM context) until setup finishes.
    param ([string]$Path)

```

```

$taskName = "ADSetupContinuation"
if (-not (Get-ScheduledTask -TaskName $taskName -ErrorAction SilentlyContinue)) {
    $action = New-ScheduledTaskAction -Execute "PowerShell.exe" -Argument "-NoProfile -
ExecutionPolicy Bypass -File `"$Path`""
    $trigger = New-ScheduledTaskTrigger -AtStartup
    $principal = New-ScheduledTaskPrincipal -UserId "SYSTEM" -LogonType ServiceAccount -RunLevel
Highest
    $settings = New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries -
StartWhenAvailable
    Register-ScheduledTask -TaskName $taskName -Action $action -Trigger $trigger -Principal
$principal -Settings $settings -Force | Out-Null
    Write-Log "Registered scheduled task '$taskName' to re-run $Path at every startup."
}
}

```

```

function Unregister-ContinuationTask {
    Unregister-ScheduledTask -TaskName "ADSetupContinuation" -Confirm:$false -ErrorAction
SilentlyContinue
}

```

```

function Get-PrimaryIPv4Info {
    # Prefer the adapter that actually has a default gateway (the "real" network interface)
    # over any secondary/NAT/loopback adapters, to avoid configuring DNS on the wrong NIC.
    $routedInterfaceIndex = (Get-NetRoute -DestinationPrefix "0.0.0.0/0" -ErrorAction SilentlyContinue |
        Sort-Object -Property RouteMetric | Select-Object -First 1).InterfaceIndex

    $ipConfig = $null
    if ($routedInterfaceIndex) {
        $ipConfig = Get-NetIPAddress -AddressFamily IPv4 -InterfaceIndex $routedInterfaceIndex -
ErrorAction SilentlyContinue |
        Where-Object { $_.IPAddress -notlike '169.254*' } | Select-Object -First 1
    }
    if (-not $ipConfig) {
        $ipConfig = Get-NetIPAddress -AddressFamily IPv4 | Where-Object {
            $_.IPAddress -notlike '169.254*' -and $_.InterfaceAlias -notlike "Loopback*"
        } | Select-Object -First 1
    }
    if (-not $ipConfig) { throw "Could not find a usable IPv4 address on this machine." }

    return [PSCustomObject]@{
        PrivateIP = $ipConfig.IPAddress
        PrefixLength = $ipConfig.PrefixLength
        Adapter = $ipConfig.InterfaceAlias
    }
}

```

```

function Get-ReverseZoneAndRecordName {
    # Computes the in-addr.arpa zone name and the record name within that zone for a
    # given IPv4 + prefix length. Assumes an octet-aligned prefix (/8, /16, /24) — for
    # non-aligned prefixes (e.g. /27) you'd normally use RFC 2317 classless delegation
    # instead, which this does not attempt to set up.
    param ([string]$IPAddress, [int]$PrefixLength)
}

```

```

$ipBytes = [System.Net.IPAddress]::Parse($IPAddress).GetAddressBytes()
$maskBytes = [byte[]](0,0,0,0)
for ($i = 0; $i -lt 4; $i++) {
    $bits = [Math]::Min(8, [Math]::Max(0, $PrefixLength - ($i * 8)))
    $maskBytes[$i] = [byte](256 - [Math]::Pow(2, 8 - $bits))
}
$networkBytes = [byte[]](0,0,0,0)
for ($i = 0; $i -lt 4; $i++) { $networkBytes[$i] = $ipBytes[$i] -band $maskBytes[$i] }
$networkID = "$($networkBytes -join '.')/$PrefixLength"

$zoneOctetsCount = [Math]::Floor($PrefixLength / 8)
if ($zoneOctetsCount -lt 1) { $zoneOctetsCount = 1 }
if ($zoneOctetsCount -gt 3) { $zoneOctetsCount = 3 }
$hostOctetsCount = 4 - $zoneOctetsCount

$zoneOctets = $networkBytes[0..($zoneOctetsCount - 1)]
[array]::Reverse($zoneOctets)
$zoneName = ($zoneOctets -join '.') + ".in-addr.arpa"

$hostOctets = $ipBytes[$zoneOctetsCount..3]
[array]::Reverse($hostOctets)
$recordName = $hostOctets -join '.'

return [PSCustomObject]@{
    NetworkID = $networkID
    ZoneName = $zoneName
    RecordName = $recordName
}
}

Register-ContinuationTask -Path $ScriptPath

# ===== Startup banner (so you can SEE it's running) =====
$currentStage = if (Test-Path $StageFile) { (Get-Content $StageFile -Raw).Trim() } else { "<none - fresh start>" }
Write-Log "=====
Write-Log "Setup script starting."
Write-Log " Running as:   $env:USERDOMAIN\$env:USERNAME"
Write-Log " Script path:  $ScriptPath"
Write-Log " Log file:     $LogFile"
Write-Log " Current stage:  $currentStage"
Write-Log " Pending reboot:  $(Test-PendingReboot)"
Write-Log "=====

# ===== Stage 1: Rename the computer, reboot =====
if (-not (Test-Path -Path $StageFile)) {
    try {
        if ($env:COMPUTERNAME -eq $NewComputerName) {
            Write-Log "STAGE 1: Computer is already named '$NewComputerName'. Skipping rename, no
reboot needed."
            Set-Content -Path $StageFile -Value "Stage1"
            $currentStage = "Stage1"
        } else {

```

```

    Write-Log "STAGE 1: Changing computer name to $NewComputerName."
    Rename-Computer -NewName $NewComputerName -Force
    Set-Content -Path $StageFile -Value "Stage1"
    Write-Log "STAGE 1: Computer renamed to $NewComputerName. Restarting now."
    Restart-Computer -Force
    exit
}
} catch {
    Write-Log "STAGE 1 ERROR: $_"
    exit 1
}
}

# ===== Stage 2: Set Administrator password =====
if ($currentStage -eq "Stage1" -and -not (Test-PendingReboot)) {
    try {
        Write-Log "STAGE 2: Checking for local 'Administrator' account."

        # Use WMI/ADSI (no dependency on the LocalAccounts module). Target the account
        # by the literal name "Administrator" as requested:
        # - exists + enabled -> just set the password
        # - exists + disabled -> enable it, then set the password
        # - doesn't exist -> create it, enable it, add to local Administrators group, set the password
        $ADS_UF_ACCOUNTDISABLE = 0x2
        $ADS_UF_DONT_EXPIRE_PASSWD = 0x10000

        $adminWmi = Get-CimInstance -ClassName Win32_UserAccount -Filter "LocalAccount = True AND
Name = 'Administrator'" -ErrorAction SilentlyContinue

        if ($adminWmi) {
            Write-Log "STAGE 2: 'Administrator' account exists (SID $($adminWmi.SID))."
            $adsidUser = [ADSI]"WinNT://$env:COMPUTERNAME/Administrator,user"

            if (($adsidUser.UserFlags.Value -band $ADS_UF_ACCOUNTDISABLE) -ne 0) {
                Write-Log "STAGE 2: Account is disabled. Enabling it."
                $adsidUser.UserFlags = $adsidUser.UserFlags.Value -band (-bnot $ADS_UF_ACCOUNTDISABLE)
                $adsidUser.SetInfo()
            } else {
                Write-Log "STAGE 2: Account is already enabled."
            }
        } else {
            Write-Log "STAGE 2: 'Administrator' account does not exist. Creating it."
            $adsicomputer = [ADSI]"WinNT://$env:COMPUTERNAME"
            $adsidUser = $adsicomputer.Create("User", "Administrator")
            $adsidUser.SetInfo()
            $adsidUser.UserFlags = ($adsidUser.UserFlags.Value -band (-bnot $ADS_UF_ACCOUNTDISABLE)) -
bor $ADS_UF_DONT_EXPIRE_PASSWD
            $adsidUser.SetInfo()
            Write-Log "STAGE 2: 'Administrator' account created and enabled."
        }
    }

    # Set the password (covers all three cases above)
    $adsidUser.SetPassword($LocalAdminPassword)
}

```

```

$adsUser.SetInfo()
Write-Log "STAGE 2: Password set for 'Administrator'."

# Ensure full admin rights: must be a member of the local Administrators group
$group = [ADSI]"WinNT://$env:COMPUTERNAME/Administrators,group"
$isMember = $false
foreach ($member in $group.Invoke("Members")) {
    if ([ADSI]$member).InvokeGet("Name") -eq "Administrator" { $isMember = $true; break }
}
if (-not $isMember) {
    Write-Log "STAGE 2: Adding 'Administrator' to the local Administrators group."
    $group.Add("WinNT://$env:COMPUTERNAME/Administrator,user")
} else {
    Write-Log "STAGE 2: 'Administrator' is already a member of the Administrators group."
}

Write-Log "STAGE 2: 'Administrator' account is enabled, in Administrators, and password set."
Set-Content -Path $StageFile -Value "Stage2"
$currentStage = "Stage2"
} catch {
    Write-Log "STAGE 2 ERROR: $_"
    exit 1
}
}

# ===== Stage 3: Point this server's DNS client at itself =====
# Since this box will host DNS locally (installed automatically with AD DS),
# it should resolve using its own IP rather than an external DNS IP.
if ($currentStage -eq "Stage2" -and -not (Test-PendingReboot)) {
    try {
        Write-Log "STAGE 3: Getting private IP address."

        $ipInfo = Get-PrimaryIPv4Info
        $privateIP = $ipInfo.PrivateIP
        $prefixLength = $ipInfo.PrefixLength
        $adapter = $ipInfo.Adapter

        Write-Log "STAGE 3: Private IP found: $privateIP/$prefixLength on adapter $adapter."

        Set-DnsClientServerAddress -InterfaceAlias $adapter -ServerAddresses $privateIP
        Write-Log "STAGE 3: DNS client on $adapter set to $privateIP (self)."

        [PSCustomObject]@{
            PrivateIP = $privateIP
            PrefixLength = $prefixLength
            Adapter = $adapter
        } | Export-Clixml -Path $NetInfoFile

        Set-Content -Path $StageFile -Value "Stage3"
        $currentStage = "Stage3"
    } catch {
        Write-Log "STAGE 3 ERROR: $_"
        exit 1
    }
}

```

```

    }
}

# ===== Stage 4: Install AD DS, promote to Domain Controller
=====
# -InstallDns:$true installs and configures the DNS Server role as part of promotion.
if ($currentStage -eq "Stage3" -and -not (Test-PendingReboot)) {
    try {
        Write-Log "STAGE 4: Installing AD DS role."
        Install-WindowsFeature -Name AD-Domain-Services -IncludeManagementTools
        Write-Log "STAGE 4: AD DS role installed."

        Write-Log "STAGE 4: Promoting server as Domain Controller for domain $DomainName."
        Import-Module ADDSDeployment

        $SafeModePassword = ConvertTo-SecureString $LocalAdminPassword -AsPlainText -Force

        # IMPORTANT: on success, Install-ADDSForest reboots the machine itself, killing this
        # process before any code after the call can run — so we pre-mark Stage4 to survive
        # that reboot. If the cmdlet throws instead (failure), the catch block below reverts
        # the stage file back to Stage3 so the next run retries promotion instead of skipping
        # straight to the reverse-DNS stage with no DC actually in place.
        Set-Content -Path $StageFile -Value "Stage4"

        try {
            Install-ADDSForest `
                -CreateDnsDelegation:$false `
                -DatabasePath "C:\Windows\NTDS" `
                -DomainMode $DomainMode `
                -DomainName $DomainName `
                -DomainNetbiosName $NetbiosName `
                -ForestMode $ForestMode `
                -InstallDns:$true `
                -LogPath "C:\Windows\NTDS" `
                -NoRebootOnCompletion:$false `
                -SysvolPath "C:\Windows\SYSVOL" `
                -SafeModeAdministratorPassword $SafeModePassword `
                -Force:$true
        } catch {
            Write-Log "STAGE 4: Install-ADDSForest threw an error, reverting stage to Stage3 for retry: $_"
            Set-Content -Path $StageFile -Value "Stage3"
            throw
        }

        Write-Log "STAGE 4: Domain Controller promotion command completed; reboot expected."
        exit
    } catch {
        Write-Log "STAGE 4 ERROR: $_"
        exit 1
    }
}

# ===== Stage 5: Automatic reverse lookup zone =====

```

```

if (($currentStage -eq "Stage4" -or $RepairReverseDns) -and -not (Test-PendingReboot)) {
    try {
        Write-Log "STAGE 5: Configuring reverse DNS lookup zone.$(if ($RepairReverseDns) { ' (manual
repair run)' })"

        $svc = Get-Service -Name DNS -ErrorAction SilentlyContinue
        $retries = 0
        while (($null -eq $svc -or $svc.Status -ne 'Running') -and $retries -lt 12) {
            Start-Sleep -Seconds 10
            $svc = Get-Service -Name DNS -ErrorAction SilentlyContinue
            $retries++
        }
        if ($null -eq $svc -or $svc.Status -ne 'Running') {
            throw "DNS Server service did not come up in time."
        }

        # Re-derive the IP fresh rather than trusting the value cached back in Stage 3 —
        # DHCP or adapter changes between stages (including across the Stage 4 reboot)
        # can make that cached value stale, which would silently create a reverse zone
        # for the wrong subnet.
        $ipInfo = Get-PrimaryIPv4Info
        $privateIP = $ipInfo.PrivateIP
        $prefixLength = $ipInfo.PrefixLength

        if (Test-Path $NetInfoFile) {
            $cached = Import-Clixml -Path $NetInfoFile
            if ($cached.PrivateIP -ne $privateIP) {
                Write-Log "STAGE 5: NOTE — cached IP from Stage 3 was $($cached.PrivateIP), current IP is
$privateIP. Using current."
            }
        }

        $zoneInfo = Get-ReverseZoneAndRecordName -IPAddress $privateIP -PrefixLength $prefixLength
        Write-Log "STAGE 5: Derived network ID: $($zoneInfo.NetworkID) -> reverse zone
'$($zoneInfo.ZoneName)', record '$($zoneInfo.RecordName)'."

        try {
            Add-DnsServerPrimaryZone -NetworkID $zoneInfo.NetworkID -ReplicationScope "Forest" -
DynamicUpdate "Secure" -ErrorAction Stop
            Write-Log "STAGE 5: Reverse lookup zone created for $($zoneInfo.NetworkID)."
        } catch {
            if ($_.Exception.Message -match "already exists") {
                Write-Log "STAGE 5: Reverse lookup zone for $($zoneInfo.NetworkID) already exists."
            } else {
                throw
            }
        }

        # Confirm the zone actually exists before relying on it — Add-DnsServerPrimaryZone
        # can otherwise fail silently in edge cases.
        $zoneCheck = Get-DnsServerZone -Name $zoneInfo.ZoneName -ErrorAction SilentlyContinue
        if (-not $zoneCheck) {
            throw "Reverse lookup zone '$($zoneInfo.ZoneName)' does not exist after creation attempt."
        }
    }
}

```

```

}

# Don't just hope dynamic registration (ipconfig /registerdns) creates the PTR record —
# explicitly create/correct it ourselves so reverse lookups work immediately.
$targetFqdn = "$env:COMPUTERNAME.$DomainName."
$existingPtr = Get-DnsServerResourceRecord -ZoneName $zoneInfo.ZoneName -Name
$zoneInfo.RecordName -RRType Ptr -ErrorAction SilentlyContinue

if ($existingPtr) {
    $currentTarget = $existingPtr.RecordData.PtrDomainName
    if ($currentTarget -ne $targetFqdn) {
        Write-Log "STAGE 5: Existing PTR record points to '$currentTarget', expected '$targetFqdn'.
Replacing it."
        Remove-DnsServerResourceRecord -ZoneName $zoneInfo.ZoneName -Name
$zoneInfo.RecordName -RRType Ptr -Force
        Add-DnsServerResourceRecordPtr -ZoneName $zoneInfo.ZoneName -Name
$zoneInfo.RecordName -PtrDomainName $targetFqdn
    } else {
        Write-Log "STAGE 5: PTR record already correct ($($zoneInfo.RecordName).
$($zoneInfo.ZoneName) -> $targetFqdn)."
    }
} else {
    Add-DnsServerResourceRecordPtr -ZoneName $zoneInfo.ZoneName -Name
$zoneInfo.RecordName -PtrDomainName $targetFqdn
    Write-Log "STAGE 5: PTR record created: $($zoneInfo.RecordName).$($zoneInfo.ZoneName) ->
$targetFqdn"
}

Set-DnsServerScavenging -ScavengingState $true -ScavengingInterval 7.00:00:00 -ApplyOnAllZones -
ErrorAction SilentlyContinue
ipconfig /registerdns | Out-Null

# Verify the reverse lookup actually resolves before declaring success.
Start-Sleep -Seconds 5
try {
    $verify = Resolve-DnsName -Name $privateIP -Type PTR -Server $privateIP -ErrorAction Stop
    Write-Log "STAGE 5: Verified reverse lookup: $privateIP -> $($verify.NameHost)"
} catch {
    Write-Log "STAGE 5 WARNING: Reverse lookup did not verify cleanly ($_). The PTR record was
written directly to the zone, so this is likely a transient DNS service delay — retest with 'nslookup
$privateIP' in a minute."
}

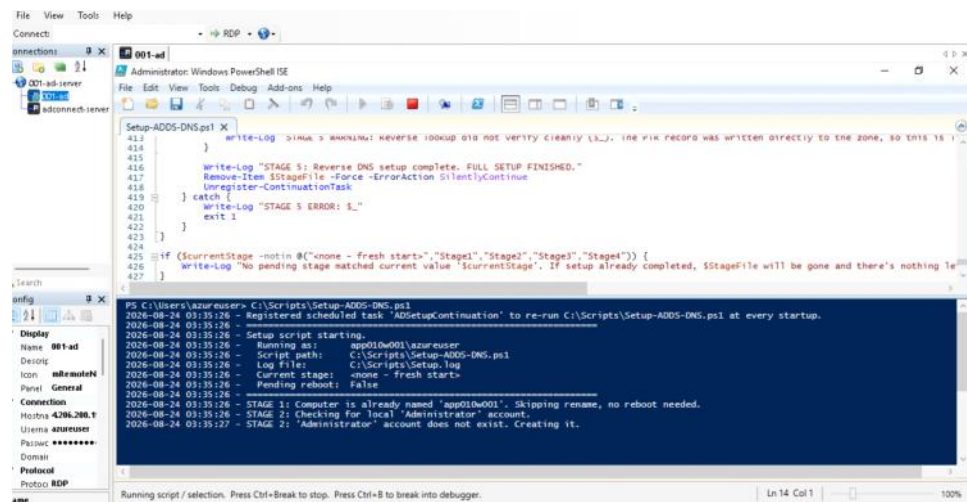
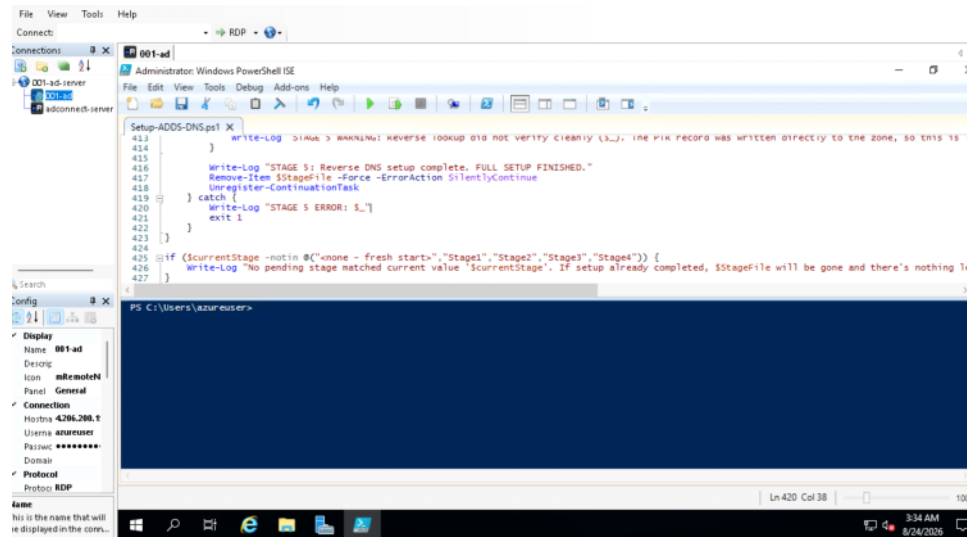
Write-Log "STAGE 5: Reverse DNS setup complete. FULL SETUP FINISHED."
Remove-Item $StageFile -Force -ErrorAction SilentlyContinue
Unregister-ContinuationTask
} catch {
    Write-Log "STAGE 5 ERROR: $_"
    exit 1
}
}

if ($currentStage -notin @("<none - fresh start>","Stage1","Stage2","Stage3","Stage4")) {

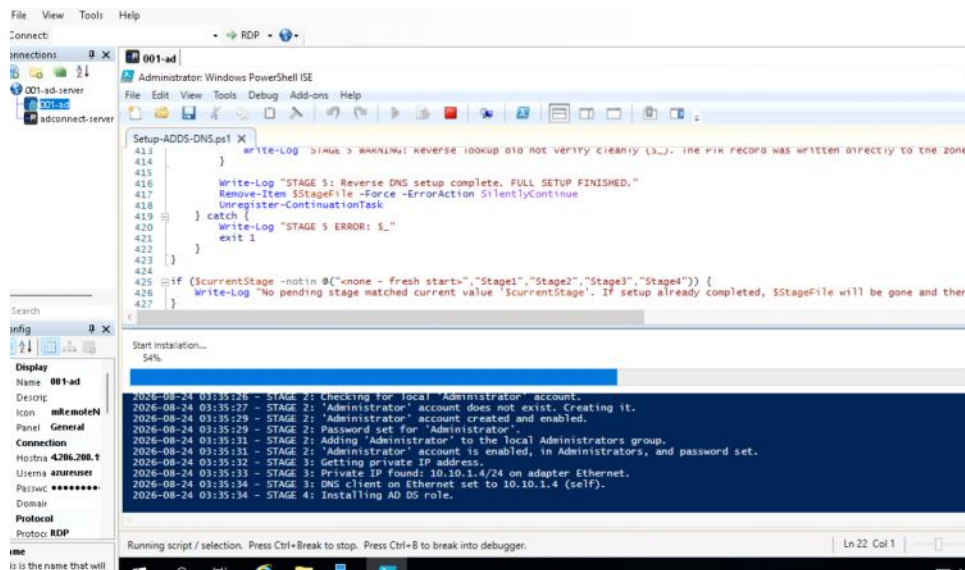
```

Write-Log "No pending stage matched current value '\$currentStage'. If setup already completed, \$StageFile will be gone and there's nothing left to do. If this looks wrong, delete \$StageFile and re-run from scratch."
}

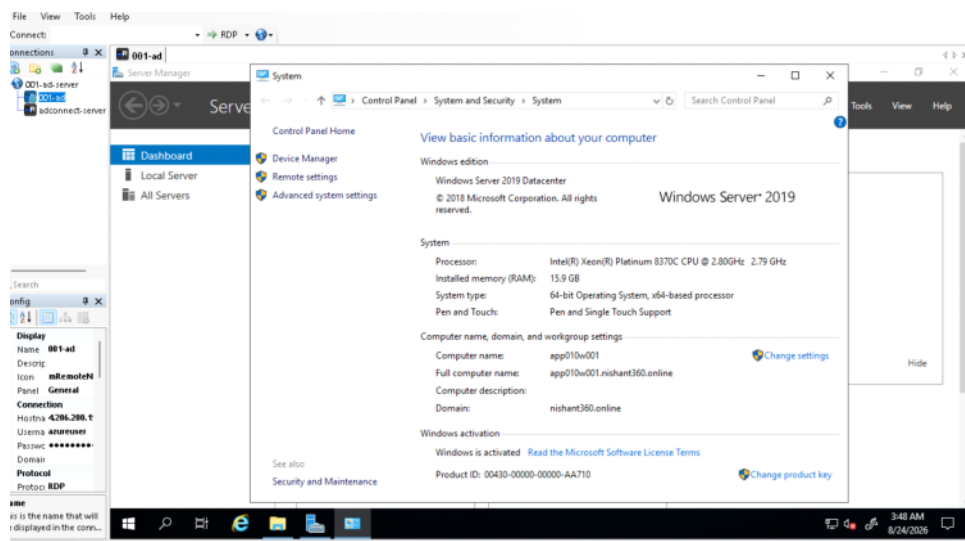
Click on green could play button



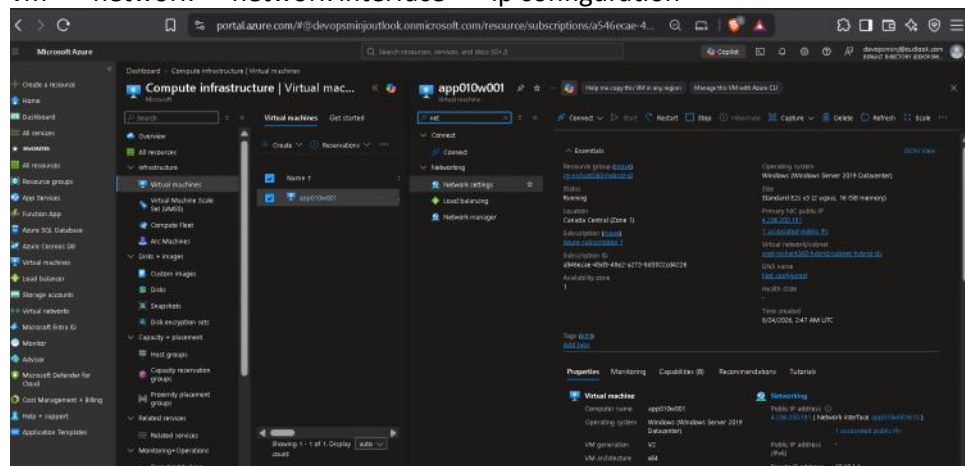
it will start the active directory server and dns server installing:



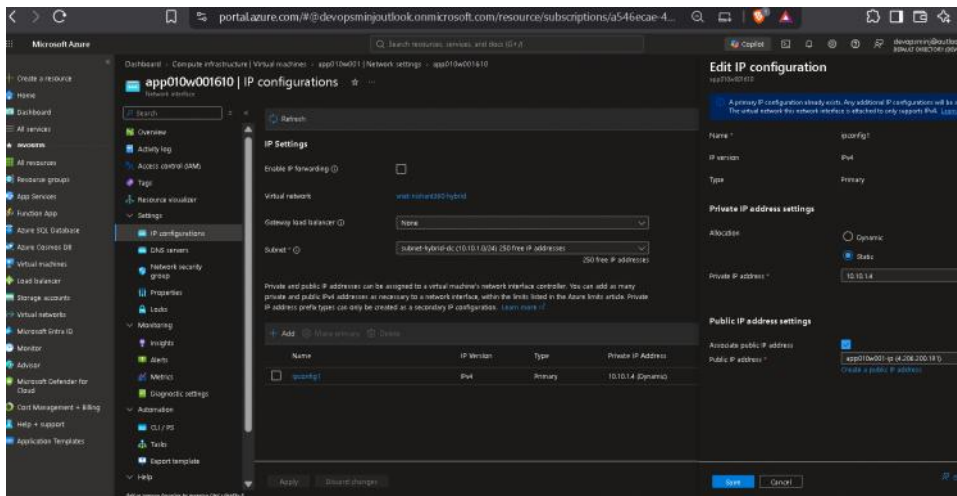
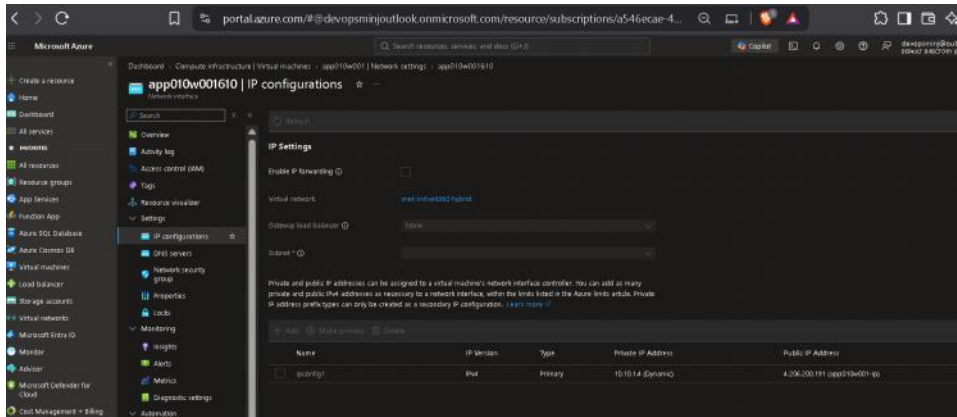
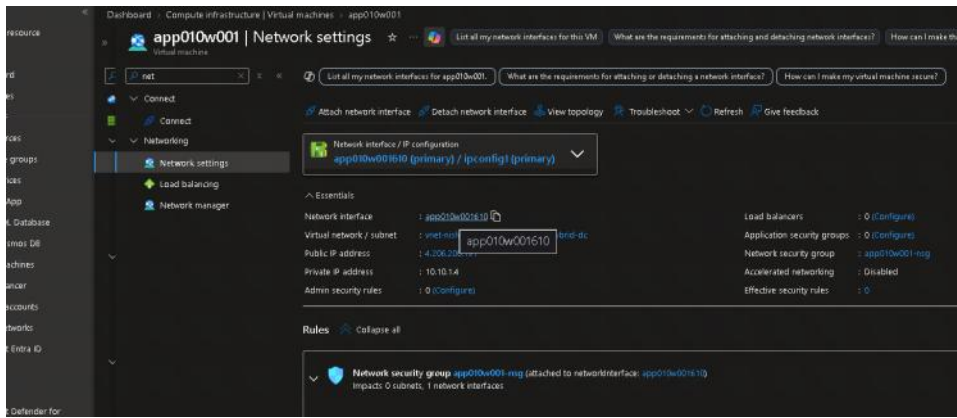
post reboot reconnect server and you will see the ad server and dns server is installed and server name changes to **full fqdn name**



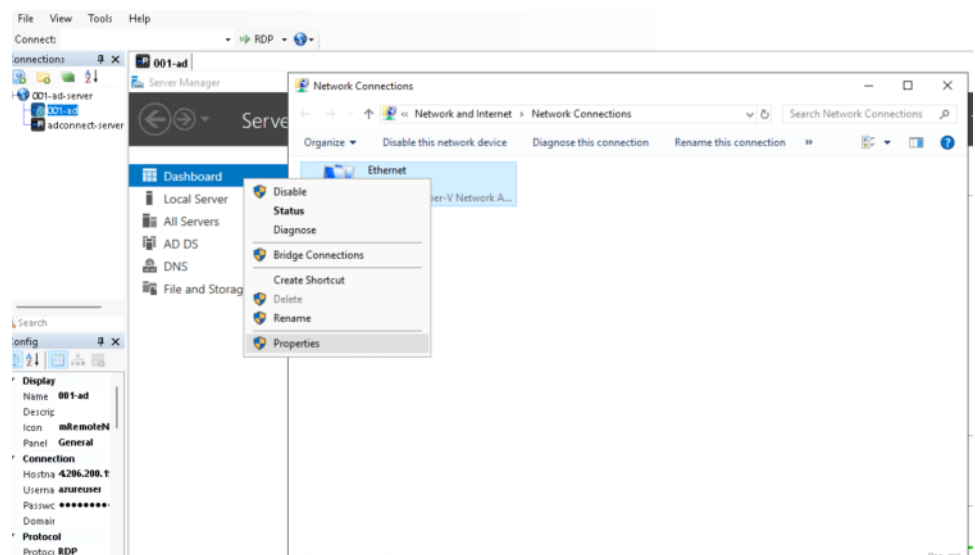
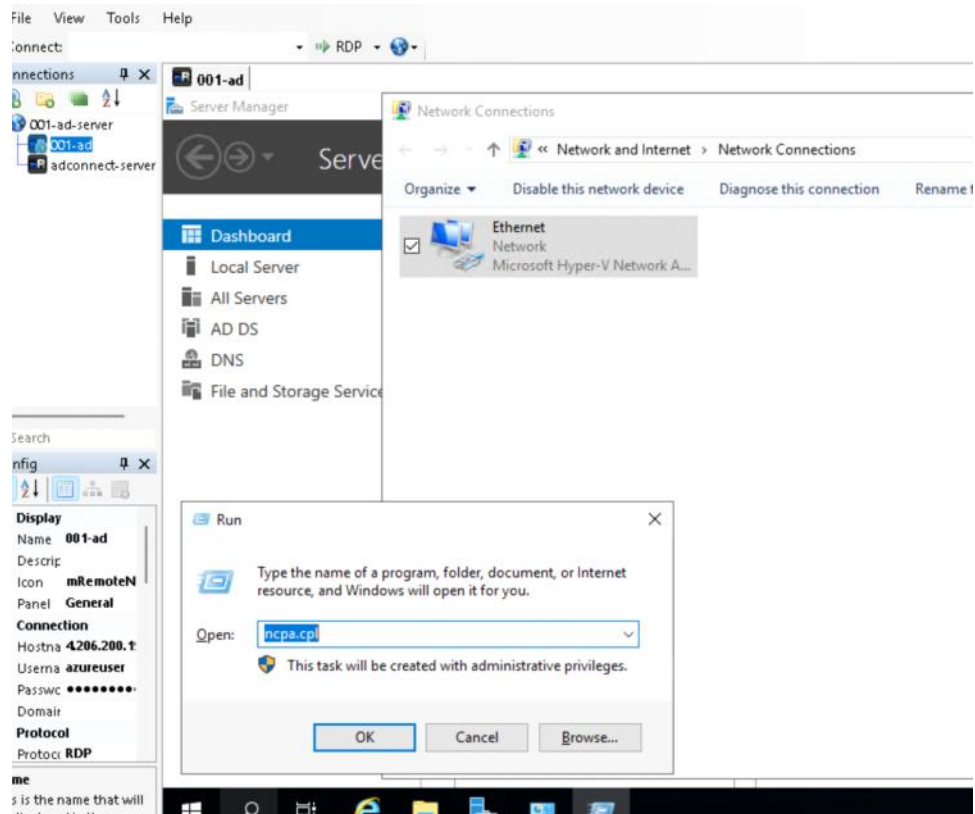
also set the static ip in azure portal in vm section meantime while its being install go to the portal --> vm --> network--> network interface --> ip configuration

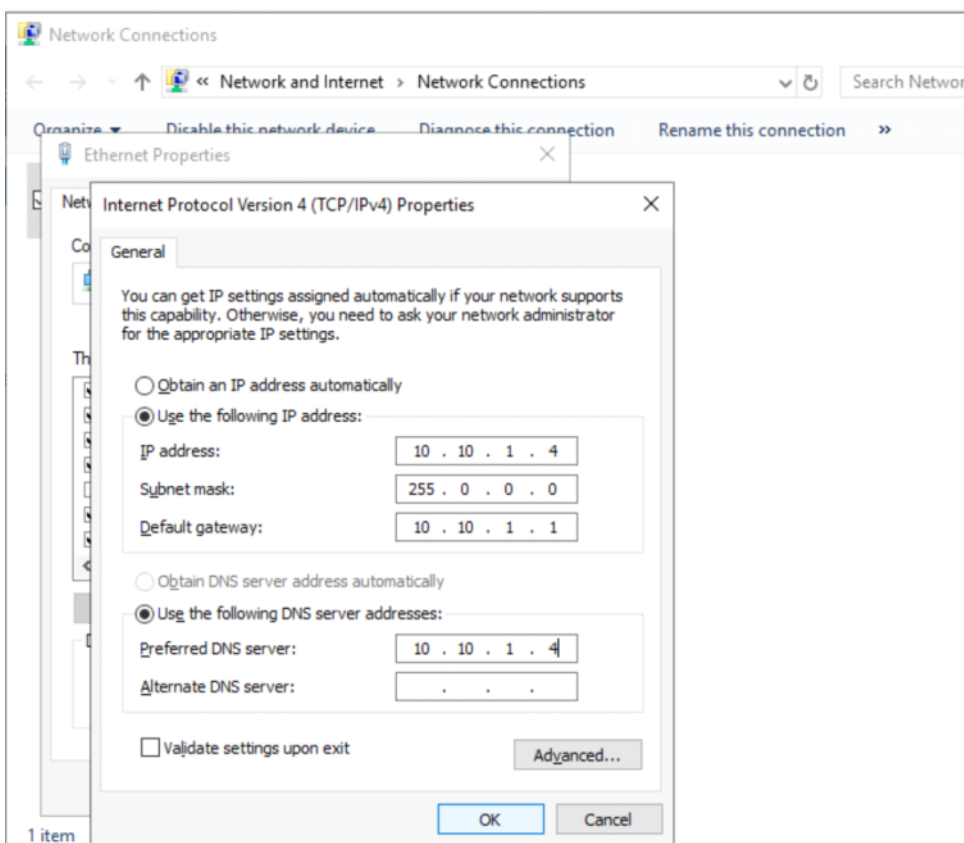
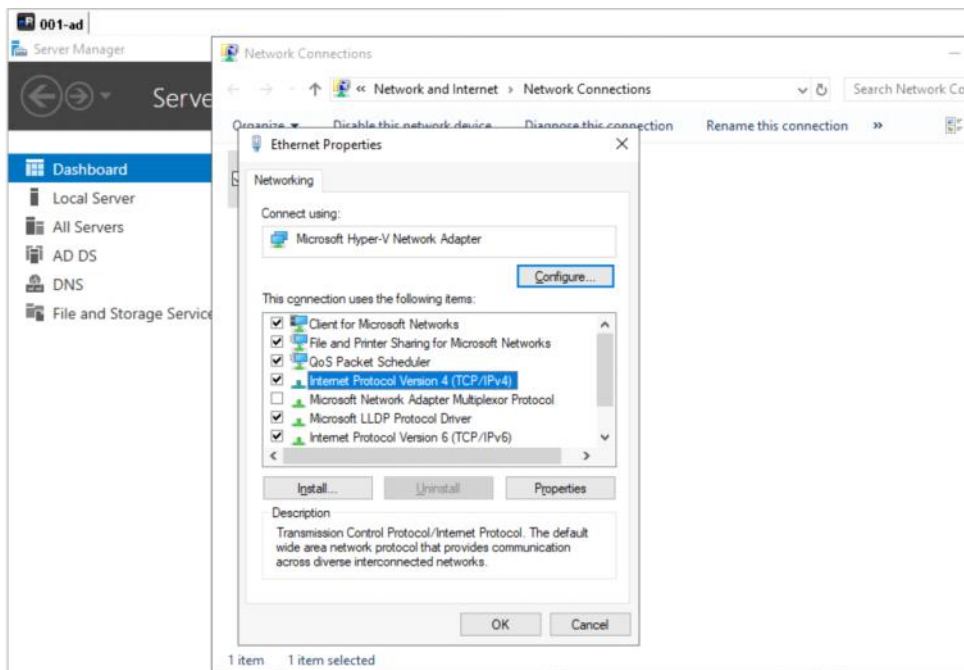


select network interface



Also set the ip address in ad server vm ip and dns ip as ad server have both ad and dns so you can use the same vm ip



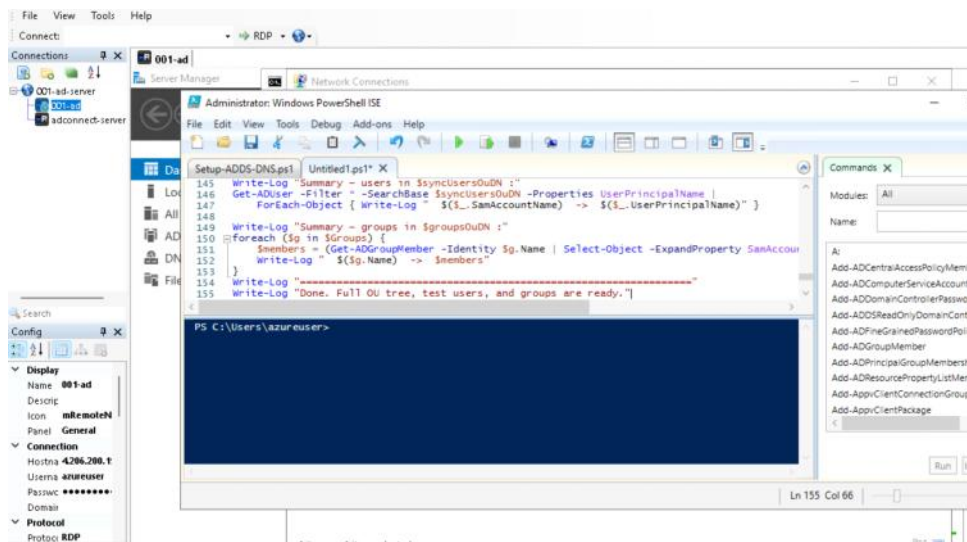
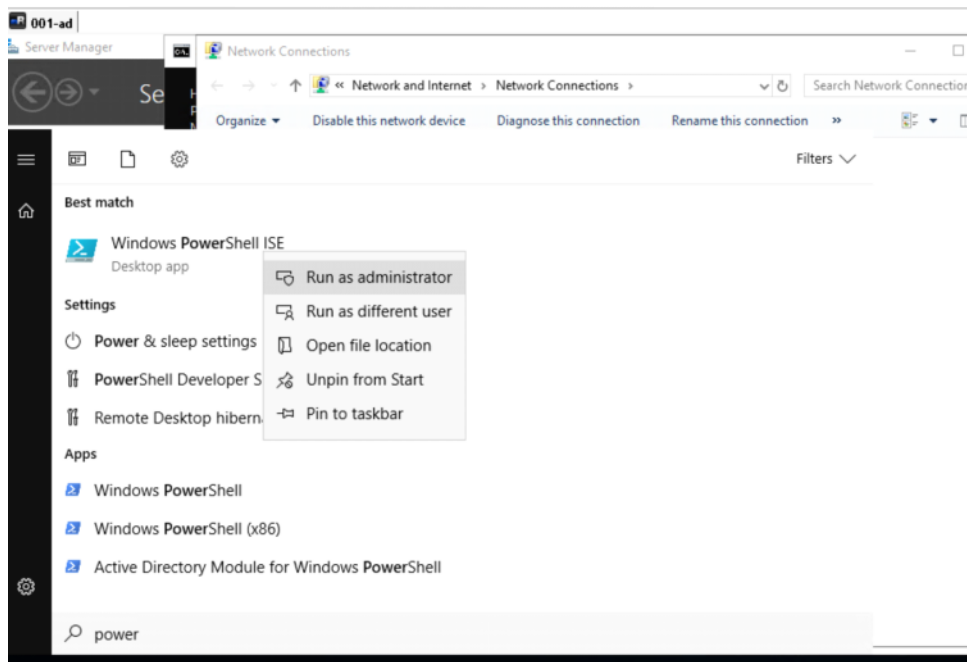


ok then click on close . There in after server will reboot this is very important . Ad and dns server is fully ready.

Now create the OU and and sample test user , OU is nothing folder structure inside that users will be placed in active directory

create the script with the same name :

C:\Scripts\New-OUStructure.ps1



```
# =====
# Builds the OU structure, test users, and security groups described in the
# Day 3 Hybrid Identity guide (Section 7): a top-level OU containing Users,
# Groups, Computers, ServiceAccounts, Admins, and SyncUsers — with SyncUsers
# being the scope Entra Connect's OU filtering will later sync to the cloud.
#
# Fully idempotent — safe to re-run. Anything that already exists is skipped
# rather than duplicated or erroring out.
#
# WHERE TO RUN THIS: on app010w001 (the Domain Controller), from an elevated
# PowerShell console, logged in as a Domain Admin. The ActiveDirectory module
# is already present on a DC — no extra install needed.
#
# SAVE THIS FILE AS: C:\Scripts\New-OUStructure.ps1
# RUN IT AS:      C:\Scripts\New-OUStructure.ps1
# =====

# ===== FILL THESE IN BEFORE RUNNING =====
```

```

$OuRoot = "MinjTech" # <<< FILL IN if you want a different top-level OU name

$SubOUs = @("Users", "Groups", "Computers", "ServiceAccounts", "Admins", "SyncUsers")

# Lab test accounts — created inside OU=SyncUsers. Edit names/logons as needed.
$TestUsers = @(
    @{ First = "Nishant"; Last = "Minj"; Logon = "nishant" }
    @{ First = "Test"; Last = "User01"; Logon = "testuser01" }
    @{ First = "Test"; Last = "User02"; Logon = "testuser02" }
    @{ First = "Cloud"; Last = "Admin"; Logon = "cloudadmin" }
)

$DefaultPassword = "ChangeMe#12345" # <<< FILL IN: password for the accounts above — LAB USE
ONLY, change this

# Security groups — created inside OU=Groups. Members reference the Logon
# values above; add/remove names as needed.
$Groups = @(
    @{ Name = "GG-Hybrid-Users"; Members = @("nishant", "testuser01", "testuser02") }
    @{ Name = "GG-DevOps"; Members = @("testuser01", "testuser02") }
    @{ Name = "GG-CloudUsers"; Members = @("nishant", "cloudadmin") }
)

$ProtectTopOuFromDeletion = $true # recommended; set $false only if you have a reason to allow
accidental deletion

# ===== Internals — no need to edit below this line =====
$errorActionPreference = "Stop"

$LogDir = "C:\Scripts"
if (-not (Test-Path $LogDir)) { New-Item -Path $LogDir -ItemType Directory -Force | Out-Null }
$LogFile = Join-Path $LogDir "New-OUStructure.log"

function Write-Log {
    param ([string]$message)
    $timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
    $line = "$timestamp - $message"
    Write-Host $line
    $line | Out-File -FilePath $LogFile -Append
}

Write-Log "=====
Write-Log "OU/User/Group build starting."
Write-Log "=====

Import-Module ActiveDirectory

$adDomain = Get-ADDomain
$domainDN = $adDomain.DistinguishedName
$upnSuffix = $adDomain.DNSRoot
Write-Log "Domain: $upnSuffix ($domainDN)"

```

```

# ----- Top-level OU -----
$topOuDN = "OU=$OuRoot,$domainDN"
if (-not (Get-ADOrganizationalUnit -Filter "Name -eq '$OuRoot'" -SearchBase $domainDN -SearchScope
OneLevel -ErrorAction SilentlyContinue)) {
    New-ADOrganizationalUnit -Name $OuRoot -Path $domainDN -ProtectedFromAccidentalDeletion:
$ProtectTopOuFromDeletion
    Write-Log "Created top-level OU: $topOuDN"
} else {
    Write-Log "Top-level OU already exists: $topOuDN"
}

# ----- Sub-OU's -----
foreach ($sub in $SubOUs) {
    if (-not (Get-ADOrganizationalUnit -Filter "Name -eq '$sub'" -SearchBase $topOuDN -SearchScope
OneLevel -ErrorAction SilentlyContinue)) {
        New-ADOrganizationalUnit -Name $sub -Path $topOuDN -ProtectedFromAccidentalDeletion:
$ProtectTopOuFromDeletion
        Write-Log "Created OU: OU=$sub,$topOuDN"
    } else {
        Write-Log "OU already exists: OU=$sub,$topOuDN"
    }
}

$syncUsersOuDN = "OU=SyncUsers,$topOuDN"
$groupsOuDN = "OU=Groups,$topOuDN"

# ----- Test users -----
$securePassword = ConvertTo-SecureString $DefaultPassword -AsPlainText -Force

foreach ($u in $TestUsers) {
    $existing = Get-ADUser -Filter "SamAccountName -eq '$($u.Logon)'" -ErrorAction SilentlyContinue
    if ($existing) {
        Write-Log "User already exists, skipping: $($u.Logon)"
        continue
    }

    New-ADUser `
        -Name "$($u.First) $($u.Last)" `
        -GivenName $u.First `
        -Surname $u.Last `
        -SamAccountName $u.Logon `
        -UserPrincipalName "$($u.Logon)@$supnSuffix" `
        -Path $syncUsersOuDN `
        -AccountPassword $securePassword `
        -Enabled $true `
        -ChangePasswordAtLogon $false `
        -PasswordNeverExpires $true # LAB ONLY — do not use PasswordNeverExpires in production

    Write-Log "Created user: $($u.Logon) ($($u.Logon)@$supnSuffix) in $syncUsersOuDN"
}

# ----- Groups -----
foreach ($g in $Groups) {

```

```

$existingGroup = Get-ADGroup -Filter "Name -eq '$($g.Name)'" -ErrorAction SilentlyContinue
if (-not $existingGroup) {
    New-ADGroup -Name $g.Name -GroupScope Global -GroupCategory Security -Path $groupsOuDN
    Write-Log "Created group: $($g.Name) in $groupsOuDN"
} else {
    Write-Log "Group already exists: $($g.Name)"
}

$currentMembers = @(Get-ADGroupMember -Identity $g.Name -ErrorAction SilentlyContinue |
Select-Object -ExpandProperty SamAccountName)
foreach ($m in $g.Members) {
    if ($currentMembers -contains $m) {
        Write-Log " $m already a member of $($g.Name)"
    } else {
        $memberExists = Get-ADUser -Filter "SamAccountName -eq '$m'" -ErrorAction SilentlyContinue
        if ($memberExists) {
            Add-ADGroupMember -Identity $g.Name -Members $m
            Write-Log " Added $m to $($g.Name)"
        } else {
            Write-Log " SKIPPED adding $m to $($g.Name) — no such user exists"
        }
    }
}
}
}

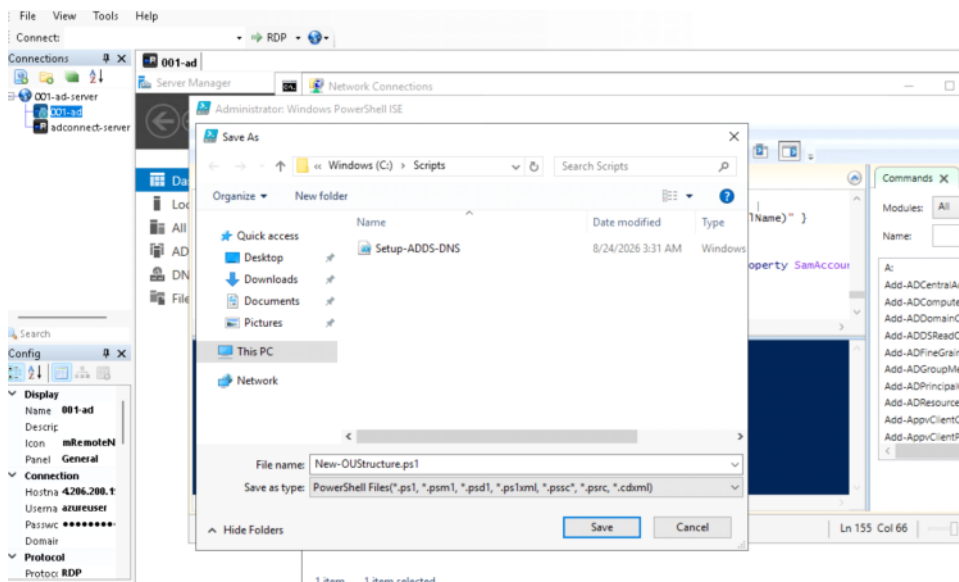
# ----- Summary -----
Write-Log "=====
Write-Log "Summary — users in $syncUsersOuDN :
Get-ADUser -Filter * -SearchBase $syncUsersOuDN -Properties UserPrincipalName |
    ForEach-Object { Write-Log " ${_.SamAccountName} -> ${_.UserPrincipalName}" }

Write-Log "Summary — groups in $groupsOuDN :
foreach ($g in $Groups) {
    $members = (Get-ADGroupMember -Identity $g.Name | Select-Object -ExpandProperty
SamAccountName) -join ", "
    Write-Log " $($g.Name) -> $members"
}
Write-Log "=====
Write-Log "Done. Full OU tree, test users, and groups are ready."

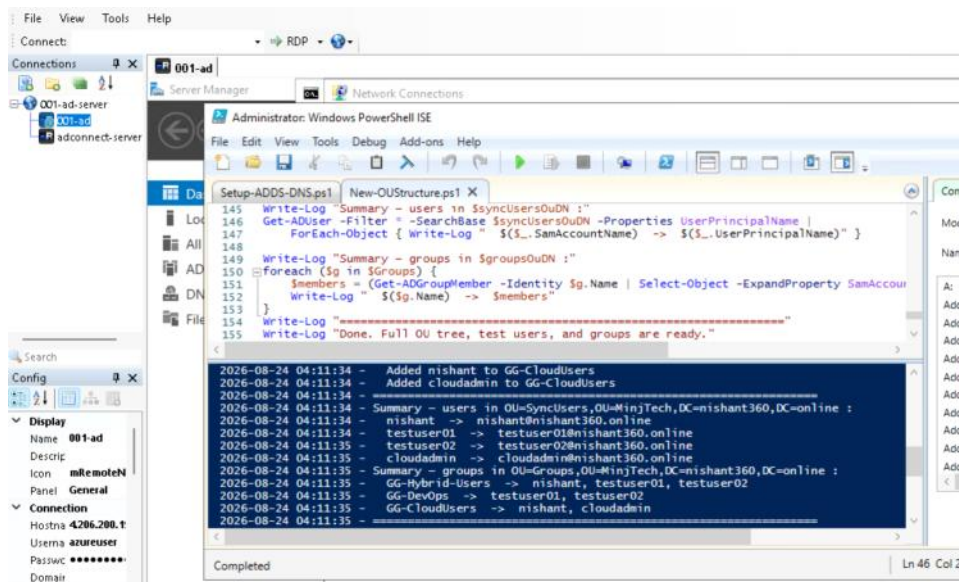
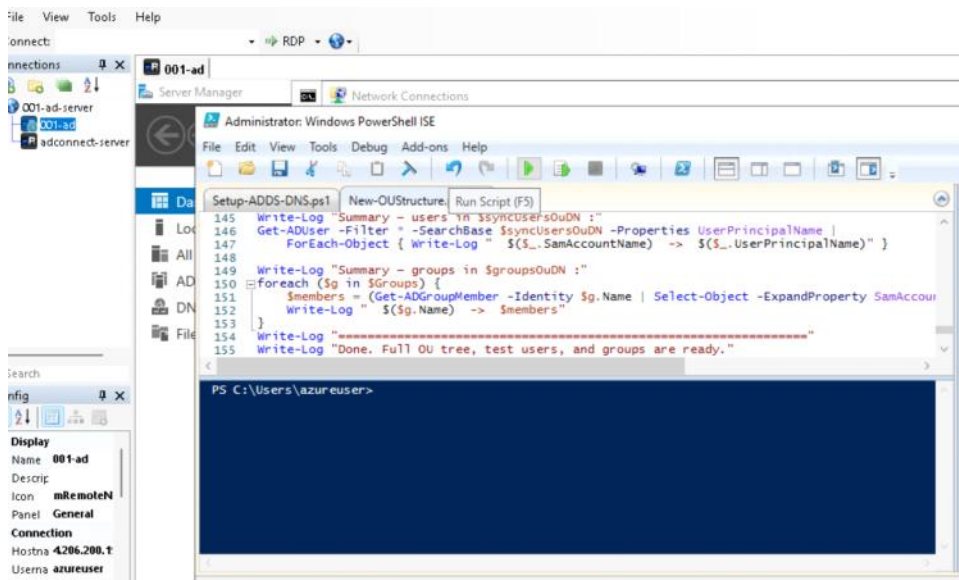
```

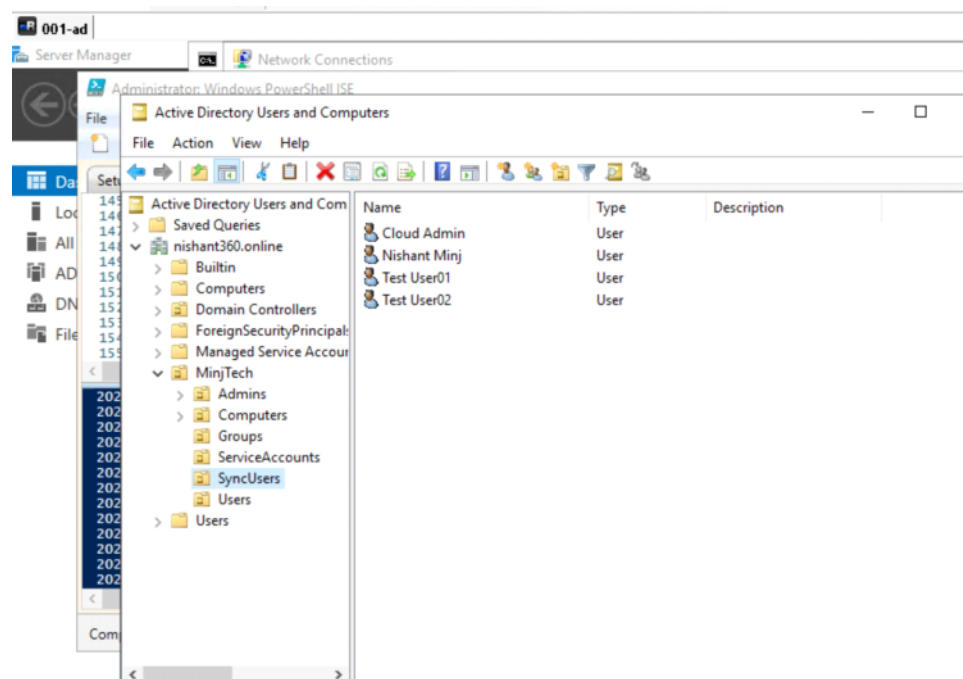
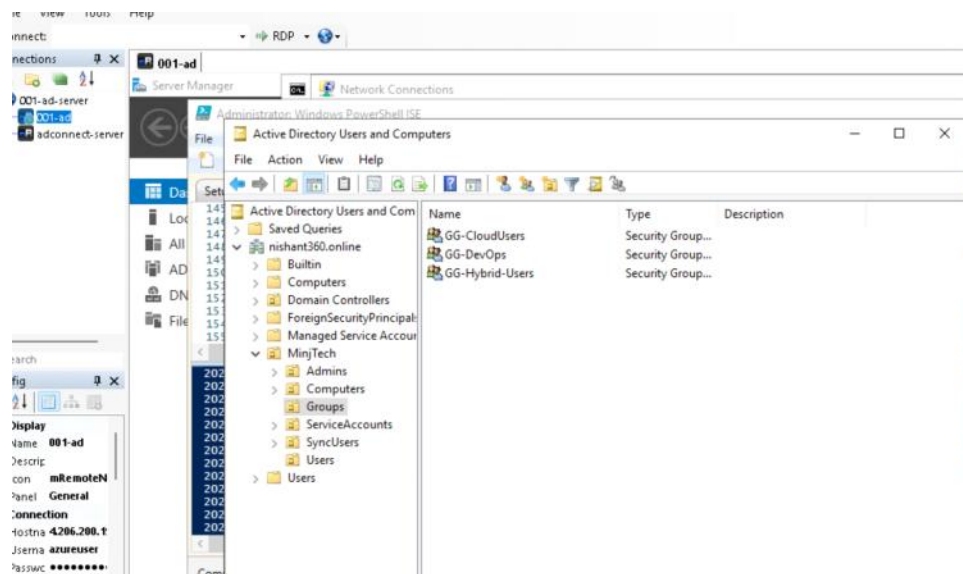
Below is the script you can download just in case

<https://github.com/minj1992/minj-active-directory/blob/main/New-OUStructure.ps1>



Execute the script:





This user and group is going to sync automatically with the azure in the last to manage the entire user organization this is big enterprise setup which we are learning.

Note: you don't have to create new vnet or subnet we are going to use existing subnet
Now we have to setup Entraid connect agent server create new vm in portal

Entra Connect server	AADCONNECT01 — 10.10.1.x
----------------------	--------------------------

Microsoft Azure

Dashboard > Compute infrastructure > Virtual machines

Create a virtual machine

Basics | Disks | Networking | Management | Monitoring | Advanced | Tags | Review + create

Create new

Instance details

Virtual machine name: AADCONNECT01

Region: (Canada) Canada Central
Deploy to an Azure Extended Zone

Availability options: Availability zone

Zone options: Self-selected zone
Select a specific zone for your VM
Azure-selected zone (Preview)
Let Azure assign the best zone for your needs

Availability zone: Zone 1

Security type: Trusted launch virtual machines
Configure security features

Image: Windows Server 2019 Datacenter - x64 Gen2
See all images | Configure VM generation

VM architecture: x64

Previous Next Review + create

Dashboard > Compute infrastructure > Virtual machines

Create a virtual machine

Basics | Disks | **Networking** | Management | Monitoring | Advanced | Tags | Review + create

This is a preview of a new Create-VM experience. Features are limited in this experience and all feedback is appreciated. [Click here to access the previous experience.](#)

Define network connectivity for your virtual machine by configuring network interface card (NIC) settings. You can control ports, inbound and outbound connectivity with security group rules, or place behind an existing load balancing solution. [Learn more](#)

Virtual network

Azure Virtual Network (VNet) enables many types of Azure resources to securely communicate with each other, the internet, and on-premises networks. [Learn more about VNets](#)

Virtual network: vnet-nishant360-hybrid

Subnet: subnet-hybrid-dc - 10.10.1.0/24

Network interface

A network interface enables an Azure virtual machine to communicate with internet, Azure, and on-premises resources. A VM can have one or more network interfaces.

Public IP: (new) AADCONNECT01p764
Create new
Public IP addresses have a nominal charge. [Learn more](#)

NIC network security group: None
Basic

Previous Next Review + create

Dashboard > CreateVirtualMachine-AADCONNECT01 | Overview

AADCONNECT01

Overview | Activity log | Access control (IAM) | Tags | Diagnose and solve problems | Monitor | Resource visualizer | Connect | Session | Windows Admin Center | Networking | Network settings | Load balancing | Application security | Network manager | Settings | Disks

⚠️ AADCONNECT01 virtual machine agent status is not ready. Troubleshoot the issue.

Connect > Start > Restart > Stop > Intermittent > Capture > Delete > Refresh > Scale > Open in mobile > Feedback > CLI / PS

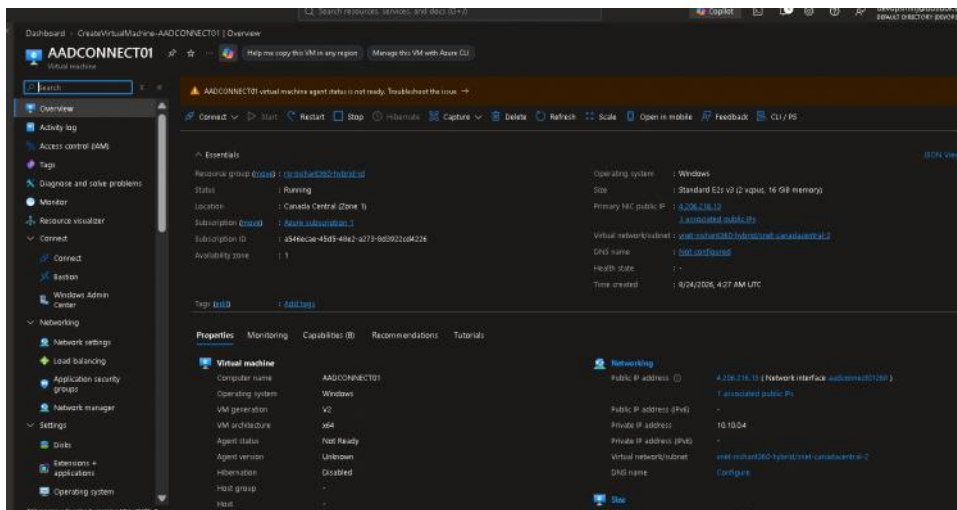
Essentials

Resource group	opsahant360-hybrid	Operating system	Windows
Status	Running	Size	Standard E3s v2 (2 vcpus, 16 GiB memory)
Location	Canada Central (Zone 1)	Primary NIC public IP	4.208.218.13
Subscription	opsahant360-1	Virtual network/virtual network	vnet-nishant360-hybrid/vnet-nishant360-hybrid
Subscription ID	4546ecce-43d1-4042-a773-9d3921d84226	DNS name	not configured
Availability zone	1	Health state	+
		Time created	9/24/2020, 4:27 AM UTC

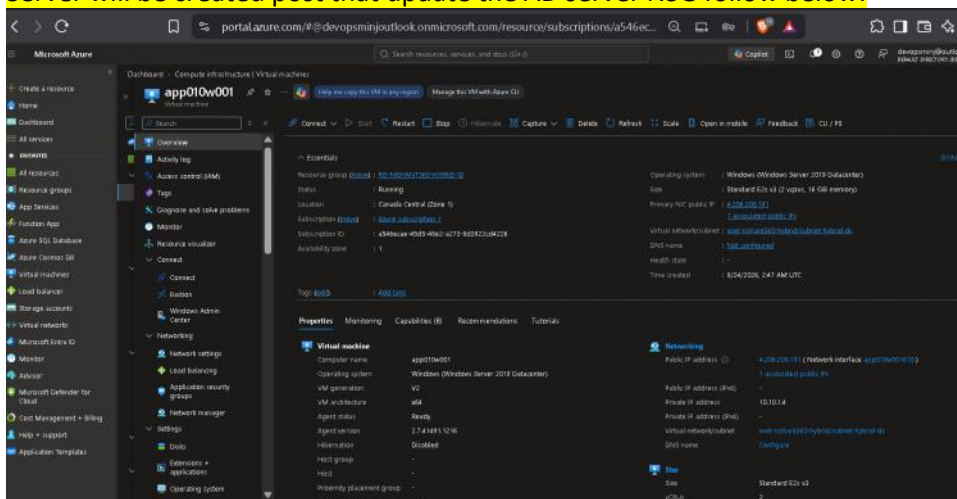
Tags: [Add tags](#)

Properties

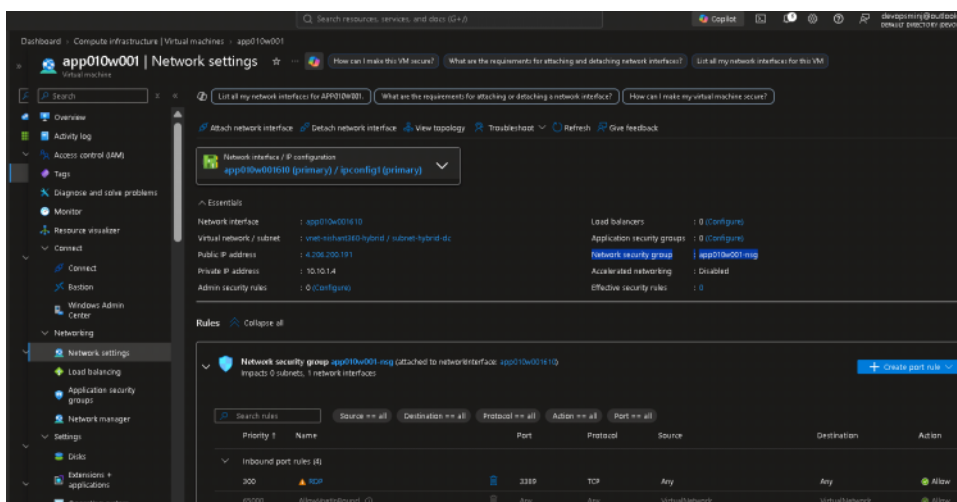
Virtual machine	AADCONNECT01	Networking	Public IP address: 4.208.218.13 (Network interface: opsahant360-hybrid-nic1)
Computer name	Windows	Public IP address (IPv4)	Associated public IP
Operating system	Windows	Public IP address (IPv6)	-
VM generation	V2	Private IP address	10.10.0.4
VM architecture	x64	Private IP address (IPv6)	-
Agent status	Not Ready		



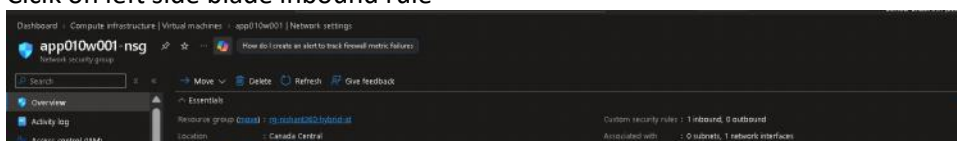
Server will be created post that update the AD server NSG follow below:

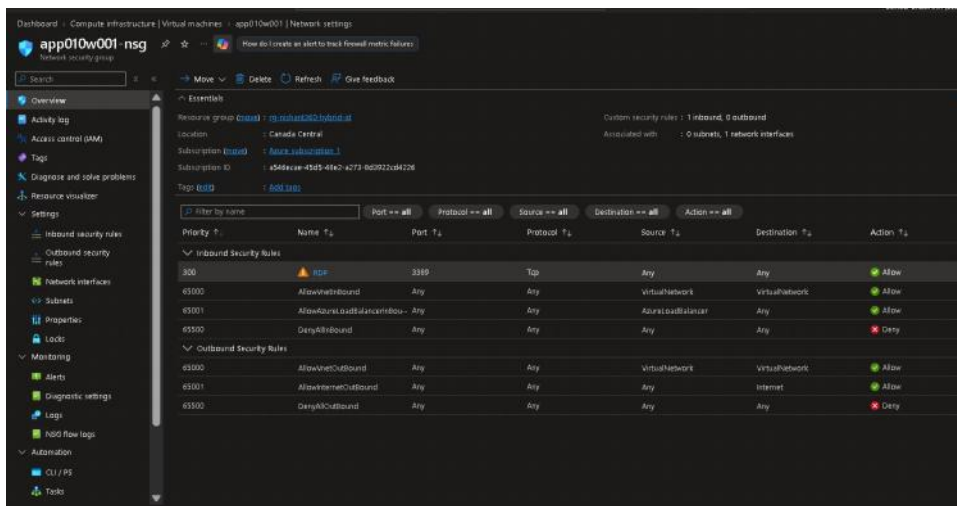


click on NSG

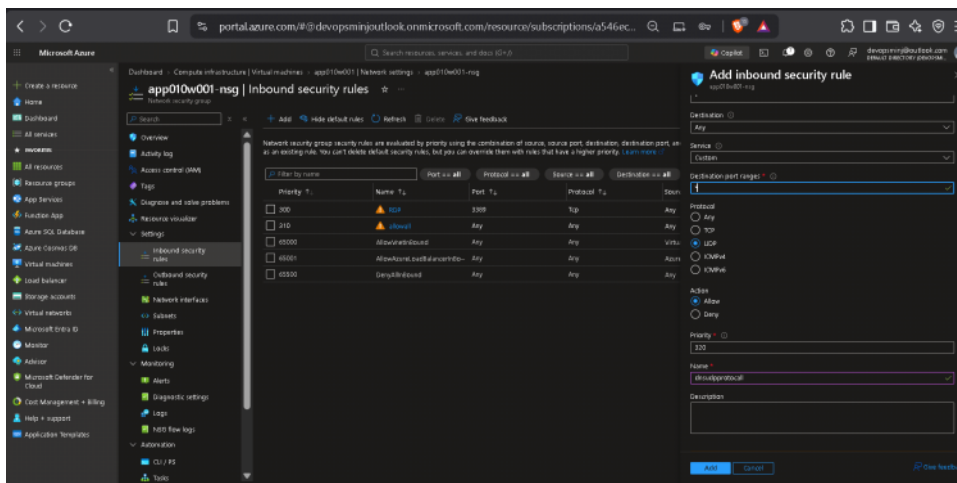
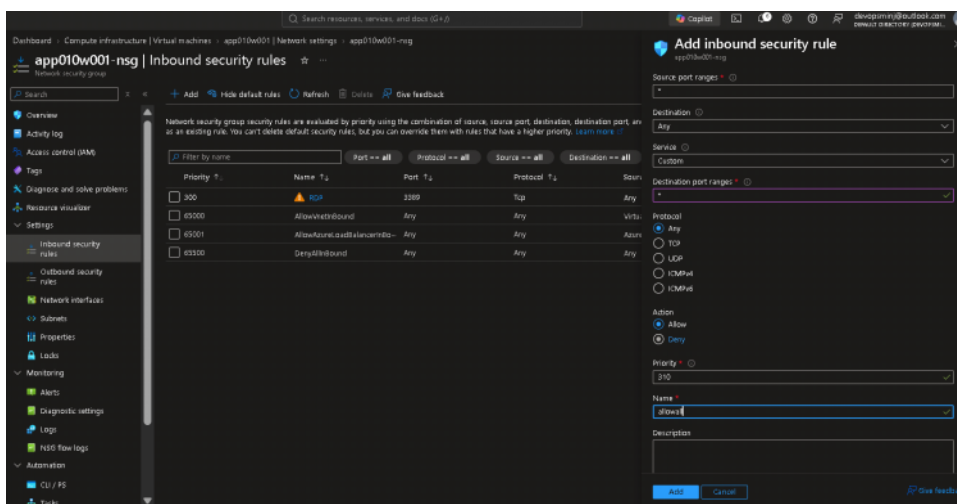


Click on left side blade inbound rule



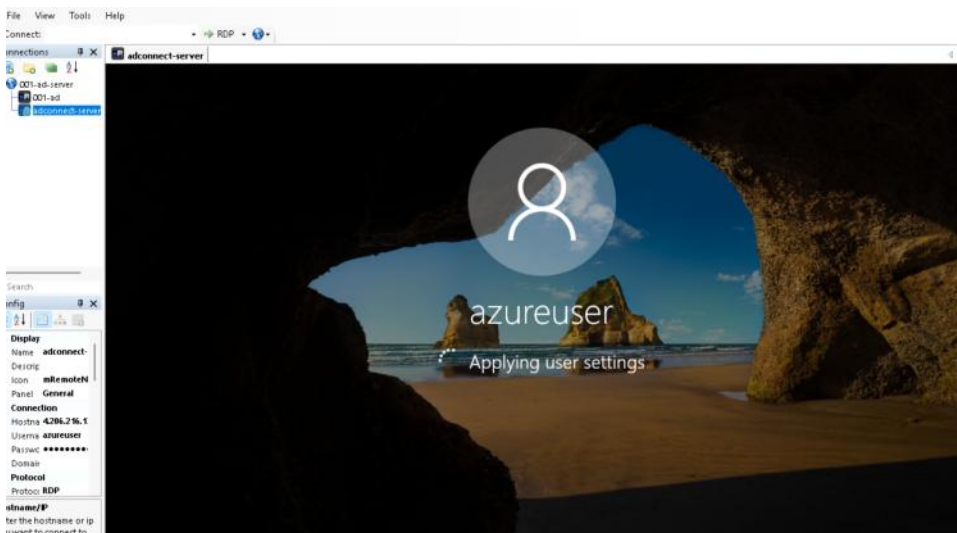
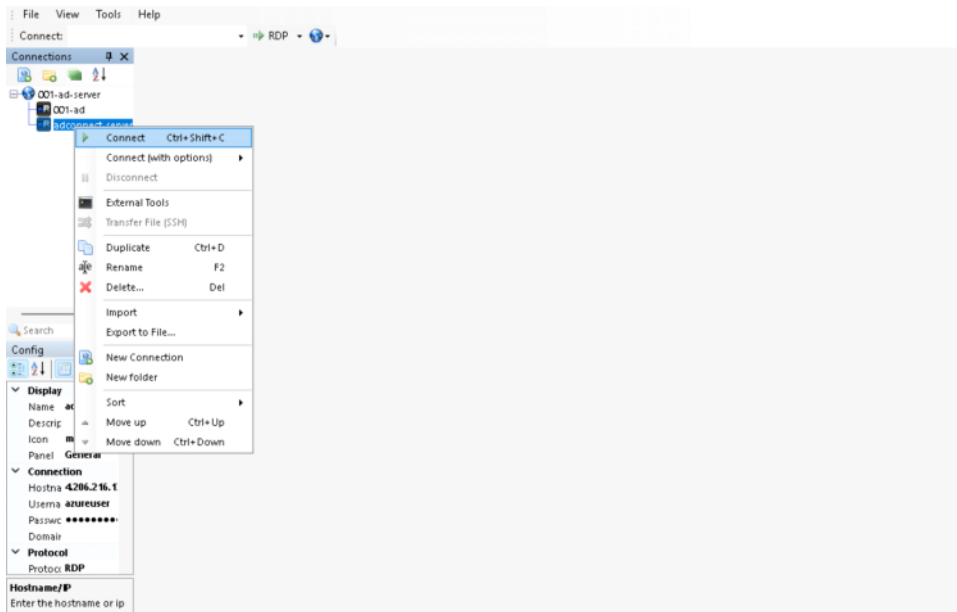


Click on add



NSG pat is completed

Login to the AADCONNECT01 vm take the public ip
 username: azureuser
 pass: Login%123456
 ip:

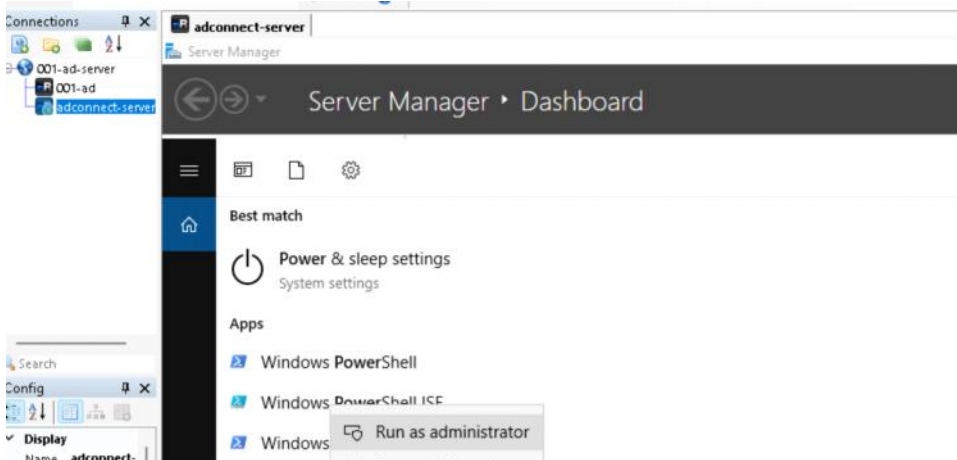


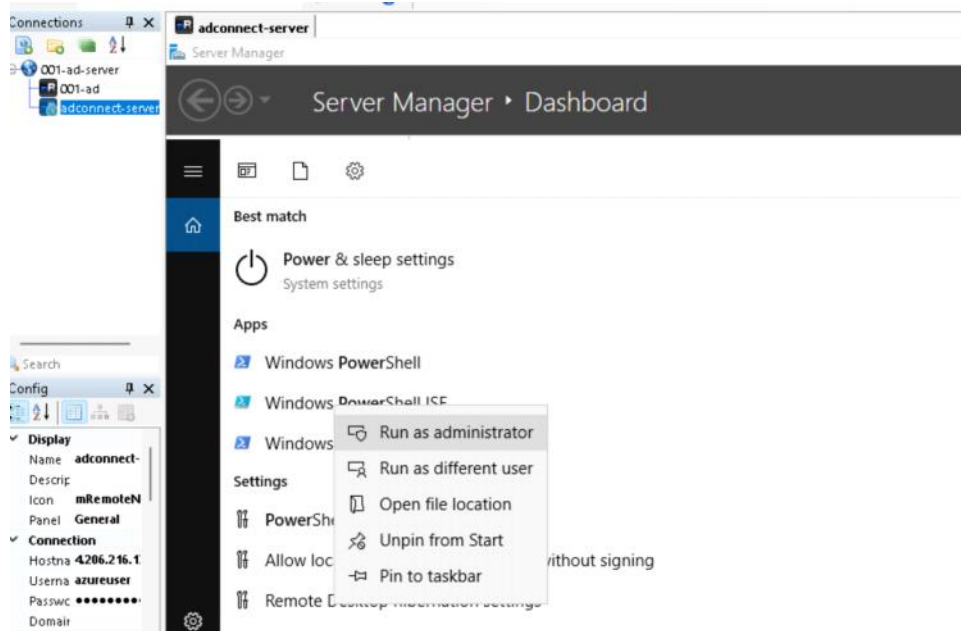
Create 2 script 2 script as given below note use AADCONNECT01 and app001w001 server private ip and dns server ip (ad server ip)

Script 1 — Join-AADCONNECT01-Domain.ps1

Save to C:\Scripts\Join-AADCONNECT01-Domain.ps1

<https://github.com/minj1992/minj-active-directory/blob/main/Join-AADCONNECT01-Domain.ps1>





Very important setting yellow color highlighted one

```
# =====
# Prepares AADCONNECT01's network (static IP + DNS pointed at the DC), verifies
# connectivity to the Domain Controller on every port Entra Connect will need,
# then domain-joins it to your AD domain. Reboot-resilient like Setup-ADDS-DNS.ps1
# — if the domain-join reboot happens, it re-launches itself and finishes the
# verification step automatically.
#
# WHERE TO RUN THIS: on AADCONNECT01 itself (not app010w001), from an elevated
# PowerShell console, AFTER AADCONNECT01 has been created as a VM and you've
# fixed app010w001's own static IP/DNS (see the earlier conversation).
#
# SAVE THIS FILE AS: C:\Scripts\Join-AADCONNECT01-Domain.ps1
# RUN IT AS:      C:\Scripts\Join-AADCONNECT01-Domain.ps1
# =====
```

```
# ===== FILL THESE IN BEFORE RUNNING =====
# Every value below MUST be checked/edited to match your actual environment
# before you run this script.
```

```
$NewComputerName = "AADCONNECT01"      # <<< FILL IN if you want a different name
$StaticIP        = "10.10.1.5"         # <<< FILL IN: a free IP in app010w001's subnet
$PrefixLength    = 24                  # <<< FILL IN: subnet prefix length (match app010w001 — /24 shown
here)
$DefaultGateway  = "10.10.1.1"         # <<< FILL IN: same default gateway as app010w001
$DnsServerIP     = "10.10.1.4"         # <<< FILL IN: app010w001's own static IP (it IS the DNS server)
$DomainFqdn      = "nishant360.online" # <<< FILL IN: your AD domain name
```

```
# You will be prompted for Domain Admin credentials interactively when this
# script reaches the domain-join step — they are never stored in this file.
```

```
# ===== Internals — no need to edit below this line =====
$ErrorActionPreference = "Stop"
```

```

$DurableScriptPath = "C:\Scripts\Join-AADCONNECT01-Domain.ps1"
$StateDir      = Split-Path -Path $DurableScriptPath -Parent
if (-not (Test-Path $StateDir)) { New-Item -Path $StateDir -ItemType Directory -Force | Out-Null }

$LogFile = Join-Path $StateDir "JoinDomain.log"
$StageFile = Join-Path $StateDir "JoinDomainStage.txt"

$CurrentPath = $MyInvocation.MyCommand.Path
if ($CurrentPath -and (Resolve-Path $CurrentPath).Path -ne (Resolve-Path -Path $DurableScriptPath -
ErrorAction SilentlyContinue).Path) {
    Copy-Item -Path $CurrentPath -Destination $DurableScriptPath -Force
} elseif (-not (Test-Path $DurableScriptPath)) {
    Write-Warning "Script has no backing file. Save this script as $DurableScriptPath and re-run it from
there so it can survive the domain-join reboot."
    exit 1
}
$ScriptPath = $DurableScriptPath

if (-not [Environment]::Is64BitProcess) {
    $sysNative = "$env:windir\Sysnative\WindowsPowerShell\v1.0\powershell.exe"
    $ps64 = if (Test-Path $sysNative) { $sysNative } else { "$env:windir\System32\WindowsPowerShell
\v1.0\powershell.exe" }
    Write-Warning "Running under 32-bit PowerShell; relaunching under $ps64 ..."
    & $ps64 -NoProfile -ExecutionPolicy Bypass -File $ScriptPath
    exit $LASTEXITCODE
}

function Write-Log {
    param ([string]$message)
    $timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
    $line = "$timestamp - $message"
    Write-Host $line
    $line | Out-File -FilePath $LogFile -Append
}

function Register-ContinuationTask {
    param ([string]$Path)
    $taskName = "AADConnectJoinContinuation"
    if (-not (Get-ScheduledTask -TaskName $taskName -ErrorAction SilentlyContinue)) {
        $action = New-ScheduledTaskAction -Execute "PowerShell.exe" -Argument "-NoProfile -
ExecutionPolicy Bypass -File `"$Path`""
        $trigger = New-ScheduledTaskTrigger -AtStartup
        $principal = New-ScheduledTaskPrincipal -UserId "SYSTEM" -LogonType ServiceAccount -RunLevel
Highest
        $settings = New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries -
StartWhenAvailable
        Register-ScheduledTask -TaskName $taskName -Action $action -Trigger $trigger -Principal
$principal -Settings $settings -Force | Out-Null
        Write-Log "Registered scheduled task '$taskName' to resume after reboot."
    }
}

function Unregister-ContinuationTask {

```

```

Unregister-ScheduledTask -TaskName "AADConnectJoinContinuation" -Confirm:$false -ErrorAction
SilentlyContinue
}

Register-ContinuationTask -Path $ScriptPath

$currentStage = if (Test-Path $StageFile) { (Get-Content $StageFile -Raw).Trim() } else { "<none - fresh
start>" }
Write-Log "=====
Write-Log "AADCONNECT01 domain-join script starting."
Write-Log " Running as: $env:USERDOMAIN\$env:USERNAME"
Write-Log " Script path: $ScriptPath"
Write-Log " Log file: $LogFile"
Write-Log " Current stage: $currentStage"
Write-Log "=====

# ===== Stage 1: Static IP, DNS, connectivity check =====
if ($currentStage -eq "<none - fresh start>") {
    try {
        Write-Log "STAGE 1: Configuring static IP $StaticIP/$PrefixLength, gateway $DefaultGateway."

        $adapter = Get-NetAdapter | Where-Object Status -eq 'Up' | Select-Object -First 1
        if (-not $adapter) { throw "No active network adapter found." }
        $ifIndex = $adapter.ifIndex

        # Remove any existing DHCP-assigned IPv4 address on this interface before adding a static one
        Get-NetIPAddress -InterfaceIndex $ifIndex -AddressFamily IPv4 -ErrorAction SilentlyContinue |
            Where-Object { $_.PrefixOrigin -eq 'Dhcp' } |
            Remove-NetIPAddress -Confirm:$false -ErrorAction SilentlyContinue

        Set-NetIPInterface -InterfaceIndex $ifIndex -Dhcp Disabled -ErrorAction SilentlyContinue

        $existing = Get-NetIPAddress -InterfaceIndex $ifIndex -AddressFamily IPv4 -ErrorAction
        SilentlyContinue |
            Where-Object { $_.IPAddress -eq $StaticIP }
        if (-not $existing) {
            New-NetIPAddress -InterfaceIndex $ifIndex -IPAddress $StaticIP -PrefixLength $PrefixLength -
            DefaultGateway $DefaultGateway | Out-Null
            Write-Log "STAGE 1: Static IP $StaticIP/$PrefixLength set on interface $ifIndex."
        } else {
            Write-Log "STAGE 1: Static IP $StaticIP already present on interface $ifIndex."
        }
    }

    Set-DnsClientServerAddress -InterfaceIndex $ifIndex -ServerAddresses $DnsServerIP
    Write-Log "STAGE 1: DNS client set to $DnsServerIP (app010w001)."

    Write-Log "STAGE 1: Verifying connectivity to the Domain Controller ($DnsServerIP) on required
    ports..."
    $ports = @(53, 88, 135, 389, 445, 464, 636)
    $failed = @()
    foreach ($port in $ports) {
        $result = Test-NetConnection -ComputerName $DnsServerIP -Port $port -WarningAction
        SilentlyContinue
    }
}

```

```

        $status = if ($result.TcpTestSucceeded) { "OK" } else { "FAILED" }
        Write-Log ("STAGE 1: Port {0,-5} -> {1}" -f $port, $status)
        if (-not $result.TcpTestSucceeded) { $failed += $port }
    }

    if ($failed.Count -gt 0) {
        throw "Connectivity check failed on port(s): $($failed -join ', '). Fix networking/NSG/firewall to
$DnsServerIP before joining the domain. Not proceeding."
    }

    Write-Log "STAGE 1: All connectivity checks passed."
    Set-Content -Path $StageFile -Value "Stage1"
    $currentStage = "Stage1"
} catch {
    Write-Log "STAGE 1 ERROR: $_"
    exit 1
}
}

# ===== Stage 2: Rename (if needed) and domain join =====
if ($currentStage -eq "Stage1") {
    try {
        if ($env:COMPUTERNAME -ne $NewComputerName) {
            Write-Log "STAGE 2: Renaming computer to $NewComputerName."
            Rename-Computer -NewName $NewComputerName -Force
            Set-Content -Path $StageFile -Value "Stage1-Renamed"
            Write-Log "STAGE 2: Renamed. Restarting before domain join."
            Restart-Computer -Force
            exit
        } else {
            Write-Log "STAGE 2: Computer already named $NewComputerName, skipping rename."
            Set-Content -Path $StageFile -Value "Stage1-Renamed"
            $currentStage = "Stage1-Renamed"
        }
    } catch {
        Write-Log "STAGE 2 ERROR: $_"
        exit 1
    }
}

if ($currentStage -eq "Stage1-Renamed") {
    try {
        Write-Log "STAGE 3: Ready to domain-join to $DomainFqdn."
        Write-Log "STAGE 3: You will be prompted for Domain Admin credentials now (not stored in this
script)."

        $cred = Get-Credential -Message "Enter Domain Admin credentials for $DomainFqdn (e.g.
$DomainFqdn\youradmin)"

        Set-Content -Path $StageFile -Value "Stage2-Joining"
        Add-Computer -DomainName $DomainFqdn -Credential $cred -Force -Restart
        # Add-Computer with -Restart reboots immediately on success; anything after this line
        # only runs if it failed without rebooting.
    }
}

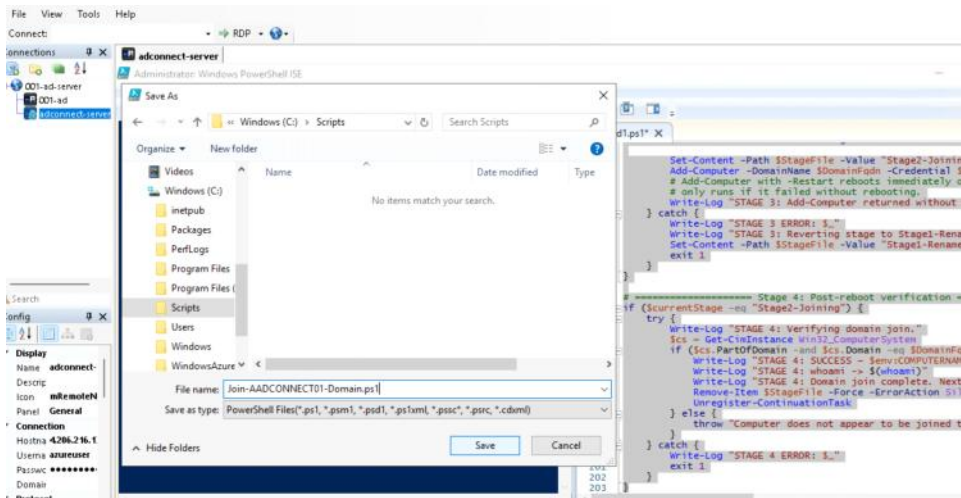
```

```

Write-Log "STAGE 3: Add-Computer returned without rebooting — check for errors above."
} catch {
    Write-Log "STAGE 3 ERROR: $_"
    Write-Log "STAGE 3: Reverting stage to Stage1-Renamed so you can retry the domain join."
    Set-Content -Path $StageFile -Value "Stage1-Renamed"
    exit 1
}
}

# ===== Stage 4: Post-reboot verification =====
if ($currentStage -eq "Stage2-Joining") {
    try {
        Write-Log "STAGE 4: Verifying domain join."
        $cs = Get-CimInstance Win32_ComputerSystem
        if ($cs.PartOfDomain -and $cs.Domain -eq $DomainFqdn) {
            Write-Log "STAGE 4: SUCCESS — $env:COMPUTERNAME is joined to $($cs.Domain)."
            Write-Log "STAGE 4: whoami -> $(whoami)"
            Write-Log "STAGE 4: Domain join complete. Next: run Prepare-EntraConnect.ps1, then follow the
manual Entra Connect wizard steps."
            Remove-Item $StageFile -Force -ErrorAction SilentlyContinue
            Unregister-ContinuationTask
        } else {
            throw "Computer does not appear to be joined to $DomainFqdn (current domain/workgroup:
$($cs.Domain))."
        }
    } catch {
        Write-Log "STAGE 4 ERROR: $_"
        exit 1
    }
}
}

```



then execute server will be rebooting , reconnect the server ad agent server


```

$NewComputerName = "AADCONNECT01"      # <<< FILL IN if you want a different name
$StaticIP        = "10.10.1.5"          # <<< FILL IN: a free IP in app010w001's subnet
$PrefixLength    = 24                   # <<< FILL IN: subnet prefix length (match app010w001 — /24 shown
here)
$DefaultGateway  = "10.10.1.1"          # <<< FILL IN: same default gateway as app010w001
$DnsServerIP     = "10.10.1.4"          # <<< FILL IN: app010w001's own static IP (it IS the DNS server)
$DomainFqdn      = "nishant360.online"   # <<< FILL IN: your AD domain name

```

```

# You will be prompted for Domain Admin credentials interactively when this
# script reaches the domain-join step — they are never stored in this file.

```

```

# ===== Internals — no need to edit below this line =====
$errorActionPreference = "Stop"

```

```

$DurableScriptPath = "C:\Scripts\Join-AADCONNECT01-Domain.ps1"
$StateDir          = Split-Path -Path $DurableScriptPath -Parent
if (-not (Test-Path $StateDir)) { New-Item -Path $StateDir -ItemType Directory -Force | Out-Null }

```

```

$LogFile = Join-Path $StateDir "JoinDomain.log"
$StageFile = Join-Path $StateDir "JoinDomainStage.txt"

```

```

$CurrentPath = $MyInvocation.MyCommand.Path
if ($CurrentPath -and (Resolve-Path $CurrentPath).Path -ne (Resolve-Path -Path $DurableScriptPath -
ErrorAction SilentlyContinue).Path) {
    Copy-Item -Path $CurrentPath -Destination $DurableScriptPath -Force
} elseif (-not (Test-Path $DurableScriptPath)) {
    Write-Warning "Script has no backing file. Save this script as $DurableScriptPath and re-run it from
there so it can survive the domain-join reboot."
    exit 1
}
$ScriptPath = $DurableScriptPath

```

```

if (-not [Environment]::Is64BitProcess) {
    $sysNative = "$env:windir\Sysnative\WindowsPowerShell\v1.0\powershell.exe"
    $ps64 = if (Test-Path $sysNative) { $sysNative } else { "$env:windir\System32\WindowsPowerShell
\v1.0\powershell.exe" }
    Write-Warning "Running under 32-bit PowerShell; relaunching under $ps64 ..."
    & $ps64 -NoProfile -ExecutionPolicy Bypass -File $ScriptPath
    exit $LASTEXITCODE
}

```

```

function Write-Log {
    param ([string]$message)
    $timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
    $line = "$timestamp - $message"
    Write-Host $line
    $line | Out-File -FilePath $LogFile -Append
}

```

```

function Register-ContinuationTask {
    param ([string]$Path)
    $taskName = "AADConnectJoinContinuation"

```

```

if (-not (Get-ScheduledTask -TaskName $taskName -ErrorAction SilentlyContinue)) {
    $action = New-ScheduledTaskAction -Execute "PowerShell.exe" -Argument "-NoProfile -
ExecutionPolicy Bypass -File `"$Path`""
    $trigger = New-ScheduledTaskTrigger -AtStartup
    $principal = New-ScheduledTaskPrincipal -UserId "SYSTEM" -LogonType ServiceAccount -RunLevel
Highest
    $settings = New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries -
StartWhenAvailable
    Register-ScheduledTask -TaskName $taskName -Action $action -Trigger $trigger -Principal
$principal -Settings $settings -Force | Out-Null
    Write-Log "Registered scheduled task '$taskName' to resume after reboot."
}
}

```

```

function Unregister-ContinuationTask {
    Unregister-ScheduledTask -TaskName "AADConnectJoinContinuation" -Confirm:$false -ErrorAction
SilentlyContinue
}

```

```
Register-ContinuationTask -Path $ScriptPath
```

```

$currentStage = if (Test-Path $StageFile) { (Get-Content $StageFile -Raw).Trim() } else { "<none - fresh
start>" }
Write-Log "=====
Write-Log "AADCONNECT01 domain-join script starting."
Write-Log " Running as: $env:USERDOMAIN\$env:USERNAME"
Write-Log " Script path: $ScriptPath"
Write-Log " Log file: $LogFile"
Write-Log " Current stage: $currentStage"
Write-Log "=====

```

```
# ===== Stage 1: Static IP, DNS, connectivity check =====
```

```

if ($currentStage -eq "<none - fresh start>") {
    try {
        Write-Log "STAGE 1: Configuring static IP $StaticIP/$PrefixLength, gateway $DefaultGateway."
    }
}

```

```

$adapter = Get-NetAdapter | Where-Object Status -eq 'Up' | Select-Object -First 1
if (-not $adapter) { throw "No active network adapter found." }
$ifIndex = $adapter.ifIndex

```

```

# Remove any existing DHCP-assigned IPv4 address on this interface before adding a static one
Get-NetIPAddress -InterfaceIndex $ifIndex -AddressFamily IPv4 -ErrorAction SilentlyContinue |
    Where-Object { $_.PrefixOrigin -eq 'Dhcp' } |
    Remove-NetIPAddress -Confirm:$false -ErrorAction SilentlyContinue

```

```
Set-NetIPInterface -InterfaceIndex $ifIndex -Dhcp Disabled -ErrorAction SilentlyContinue
```

```

$existing = Get-NetIPAddress -InterfaceIndex $ifIndex -AddressFamily IPv4 -ErrorAction
SilentlyContinue |
    Where-Object { $_.IPAddress -eq $StaticIP }
if (-not $existing) {
    New-NetIPAddress -InterfaceIndex $ifIndex -IPAddress $StaticIP -PrefixLength $PrefixLength -
DefaultGateway $DefaultGateway | Out-Null
}

```

```

        Write-Log "STAGE 1: Static IP $StaticIP/$PrefixLength set on interface $ifIndex."
    } else {
        Write-Log "STAGE 1: Static IP $StaticIP already present on interface $ifIndex."
    }

    Set-DnsClientServerAddress -InterfaceIndex $ifIndex -ServerAddresses $DnsServerIP
    Write-Log "STAGE 1: DNS client set to $DnsServerIP (app010w001)."
```

Write-Log "STAGE 1: Verifying connectivity to the Domain Controller (\$DnsServerIP) on required ports..."

```

    $ports = @(53, 88, 135, 389, 445, 464, 636)
    $failed = @()
    foreach ($port in $ports) {
        $result = Test-NetConnection -ComputerName $DnsServerIP -Port $port -WarningAction
        SilentlyContinue
        $status = if ($result.TcpTestSucceeded) { "OK" } else { "FAILED" }
        Write-Log ("STAGE 1: Port {0,-5} -> {1}" -f $port, $status)
        if (-not $result.TcpTestSucceeded) { $failed += $port }
    }

    if ($failed.Count -gt 0) {
        throw "Connectivity check failed on port(s): $($failed -join ', '). Fix networking/NSG/firewall to
        $DnsServerIP before joining the domain. Not proceeding."
    }

    Write-Log "STAGE 1: All connectivity checks passed."
    Set-Content -Path $StageFile -Value "Stage1"
    $currentStage = "Stage1"
} catch {
    Write-Log "STAGE 1 ERROR: $_"
    exit 1
}
}

# ===== Stage 2: Rename (if needed) and domain join =====
if ($currentStage -eq "Stage1") {
    try {
        if ($env:COMPUTERNAME -ne $NewComputerName) {
            Write-Log "STAGE 2: Renaming computer to $NewComputerName."
            Rename-Computer -NewName $NewComputerName -Force
            Set-Content -Path $StageFile -Value "Stage1-Renamed"
            Write-Log "STAGE 2: Renamed. Restarting before domain join."
            Restart-Computer -Force
            exit
        } else {
            Write-Log "STAGE 2: Computer already named $NewComputerName, skipping rename."
            Set-Content -Path $StageFile -Value "Stage1-Renamed"
            $currentStage = "Stage1-Renamed"
        }
    } catch {
        Write-Log "STAGE 2 ERROR: $_"
        exit 1
    }
}

```

```

}

if ($currentStage -eq "Stage1-Renamed") {
    try {
        Write-Log "STAGE 3: Ready to domain-join to $DomainFqdn."
        Write-Log "STAGE 3: You will be prompted for Domain Admin credentials now (not stored in this script)."
```

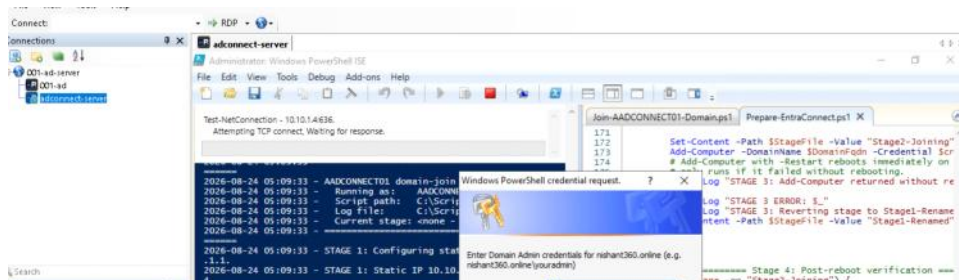
 \$cred = Get-Credential -Message "Enter Domain Admin credentials for \$DomainFqdn (e.g. \$DomainFqdn\youradmin)"

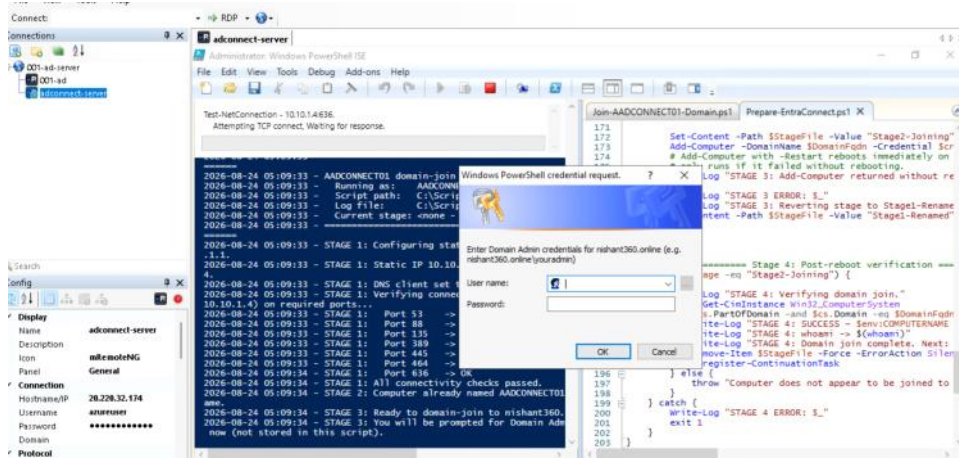
```

        Set-Content -Path $StageFile -Value "Stage2-Joining"
        Add-Computer -DomainName $DomainFqdn -Credential $cred -Force -Restart
        # Add-Computer with -Restart reboots immediately on success; anything after this line
        # only runs if it failed without rebooting.
        Write-Log "STAGE 3: Add-Computer returned without rebooting — check for errors above."
    } catch {
        Write-Log "STAGE 3 ERROR: $_"
        Write-Log "STAGE 3: Reverting stage to Stage1-Renamed so you can retry the domain join."
        Set-Content -Path $StageFile -Value "Stage1-Renamed"
        exit 1
    }
}

# ===== Stage 4: Post-reboot verification =====
if ($currentStage -eq "Stage2-Joining") {
    try {
        Write-Log "STAGE 4: Verifying domain join."
        $cs = Get-CimInstance Win32_ComputerSystem
        if ($cs.PartOfDomain -and $cs.Domain -eq $DomainFqdn) {
            Write-Log "STAGE 4: SUCCESS — $env:COMPUTERNAME is joined to $($cs.Domain)."
            Write-Log "STAGE 4: whoami -> $(whoami)"
            Write-Log "STAGE 4: Domain join complete. Next: run Prepare-EntraConnect.ps1, then follow the manual Entra Connect wizard steps."
            Remove-Item $StageFile -Force -ErrorAction SilentlyContinue
            Unregister-ContinuationTask
        } else {
            throw "Computer does not appear to be joined to $DomainFqdn (current domain/workgroup: $($cs.Domain))."
        }
    } catch {
        Write-Log "STAGE 4 ERROR: $_"
        exit 1
    }
}
}

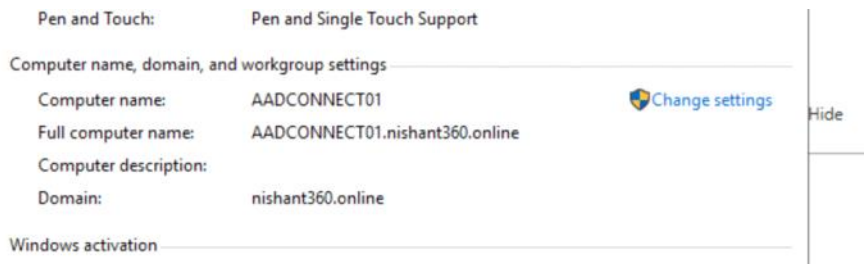
```



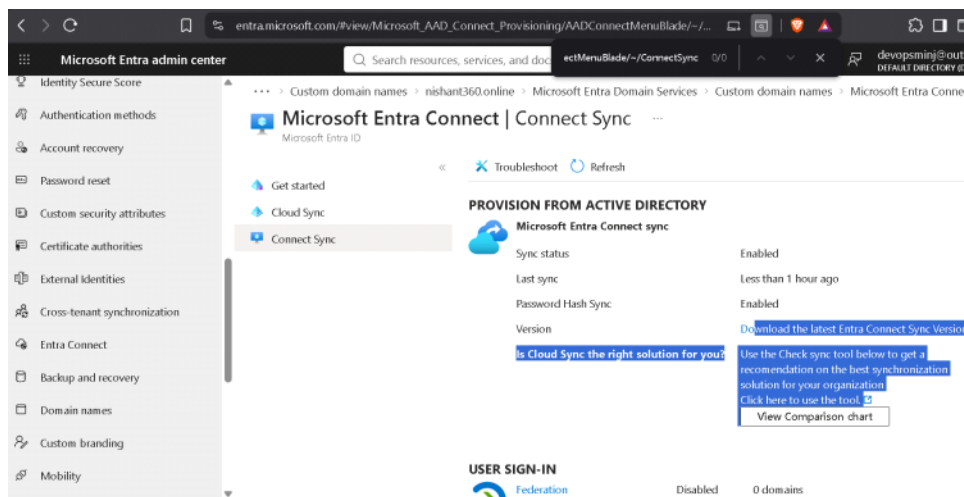


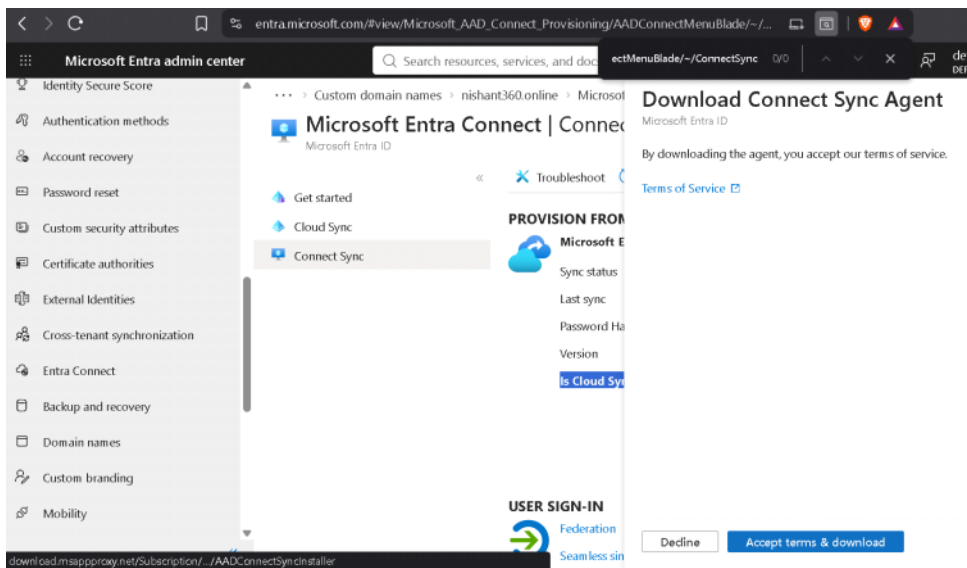
put the username: azureuser
pass: Login%123456

There in after computer will reboot then reconnect the server, now you can see the server ad conenct is domain joined

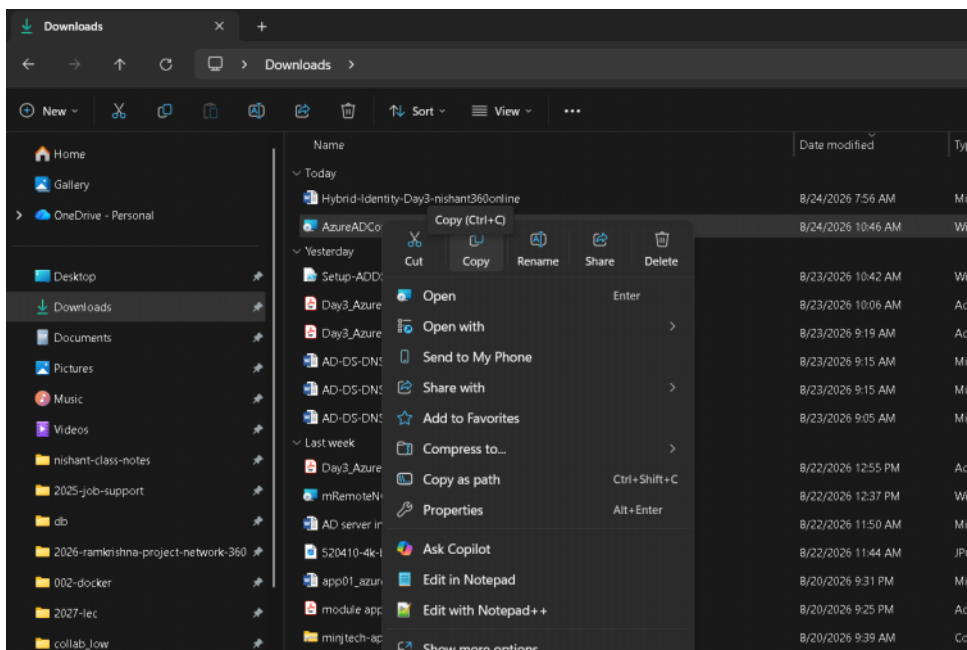
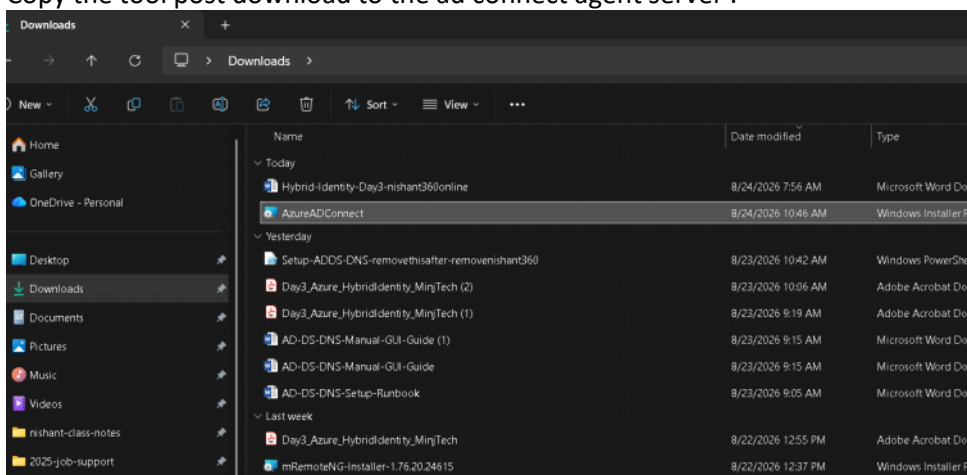


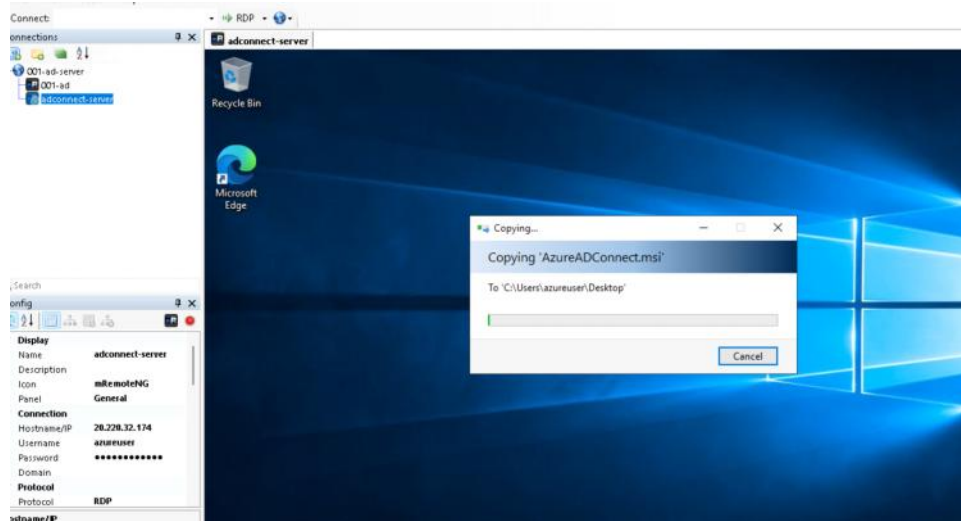
Now install the ad sync agent in ad agent sync server , down load the tool
https://entra.microsoft.com/#view/Microsoft_AAD_Connect_Provisioning/AADConnectMenuBlade/~/_/ConnectSync



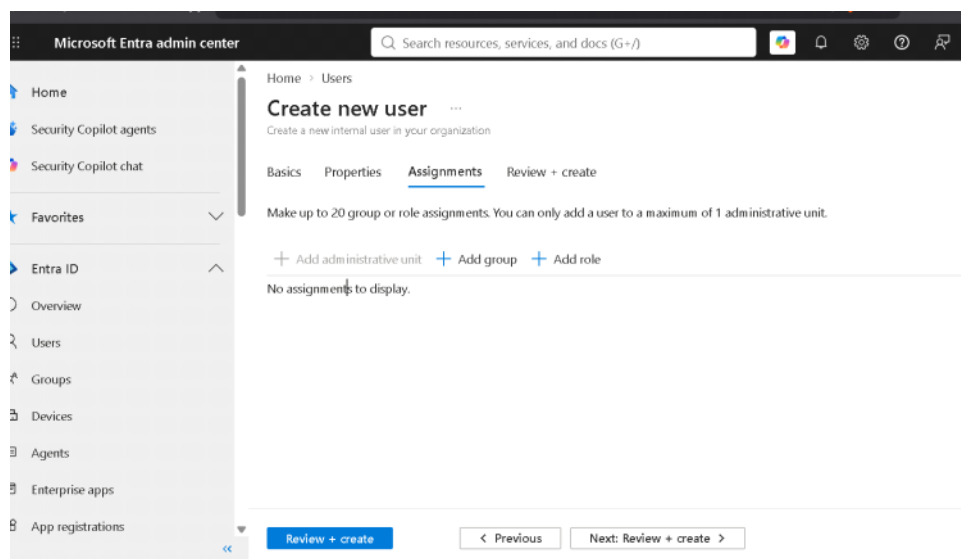
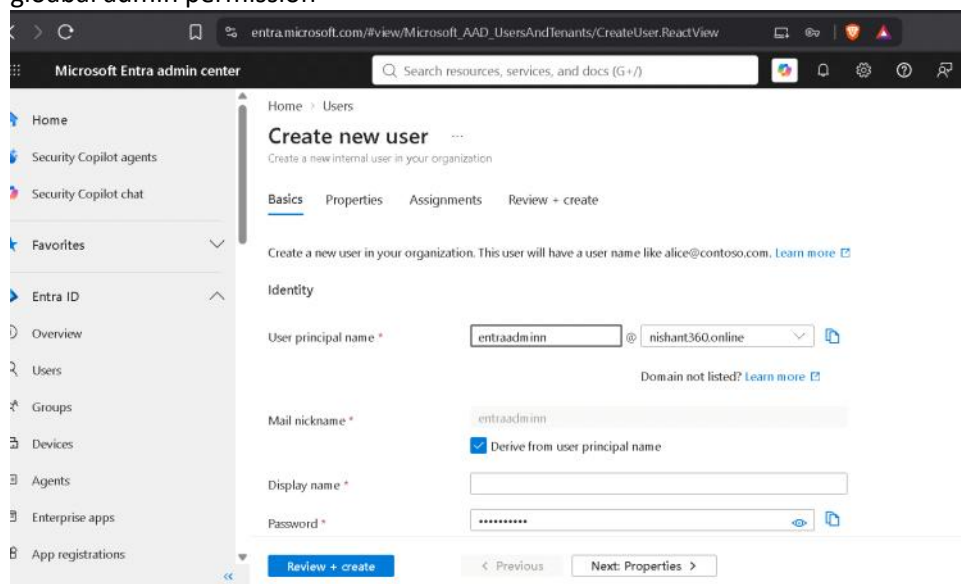


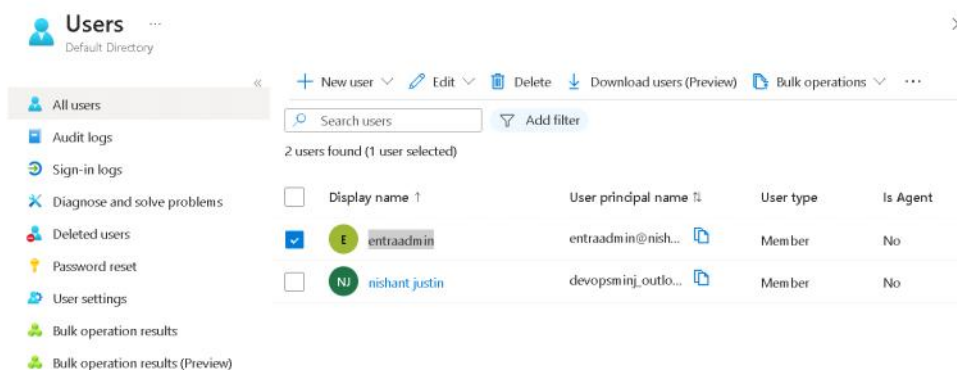
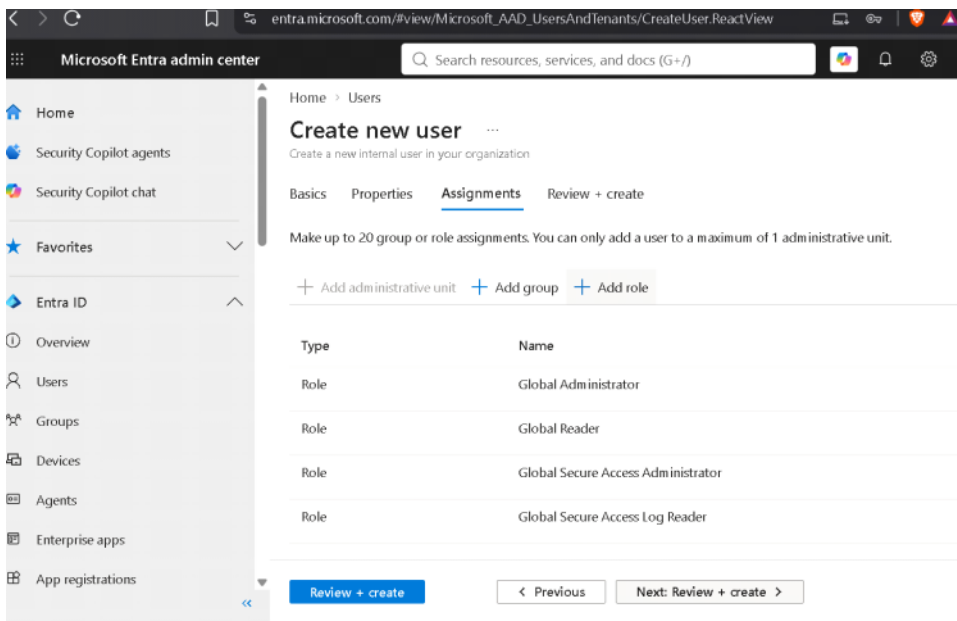
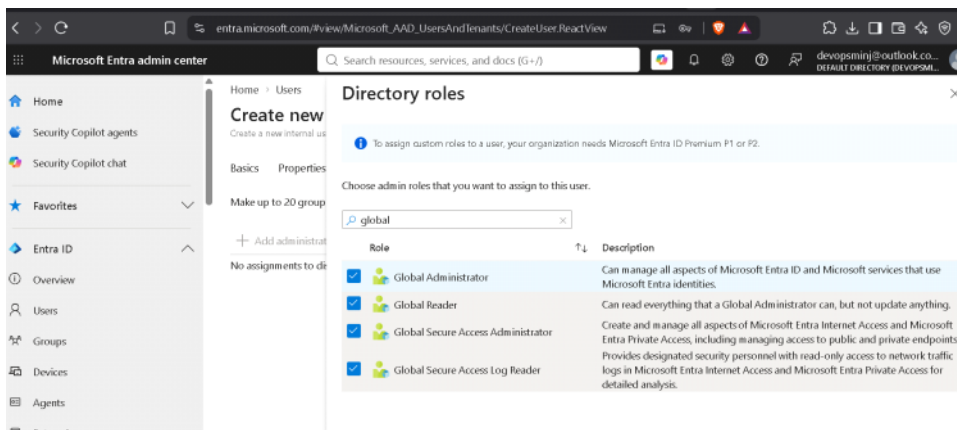
Copy the tool post download to the ad connect agent server :





meanwhile create the `entraadmin@nishant360.online` replace with your domain name and give the global admin permission



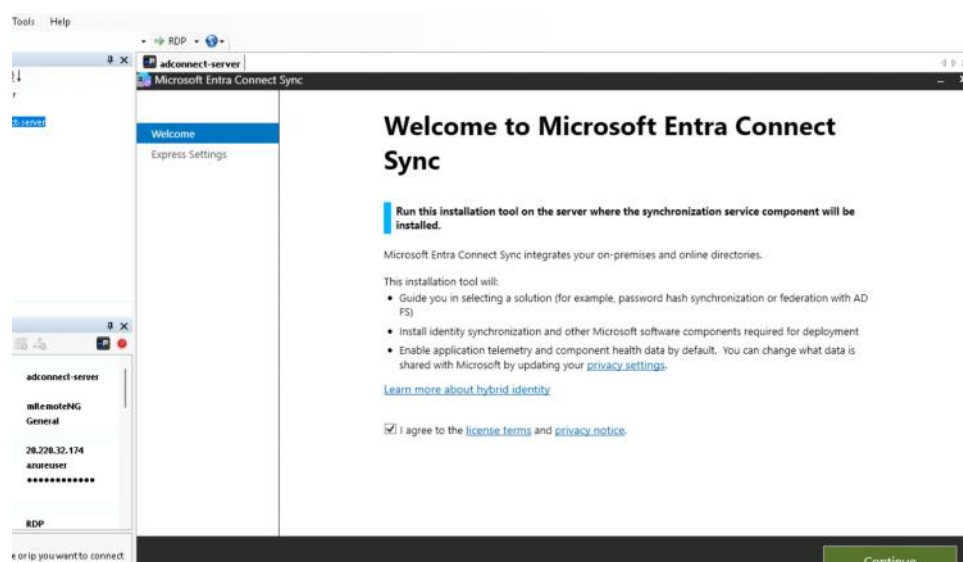
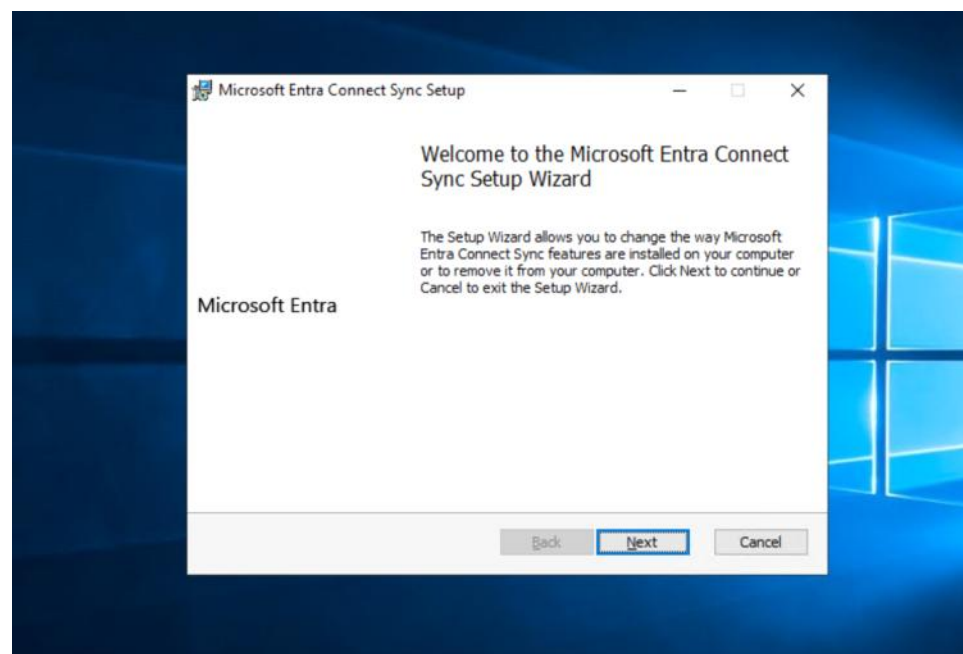
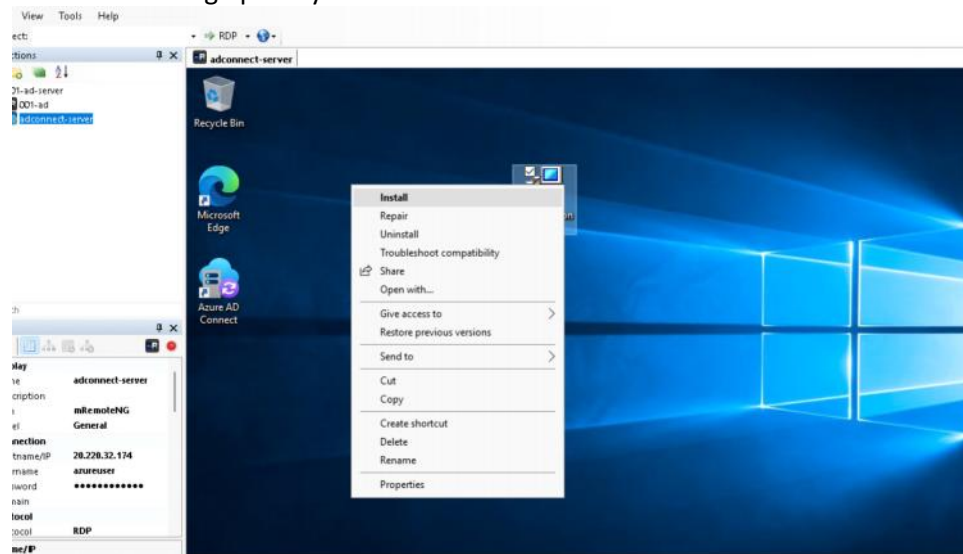


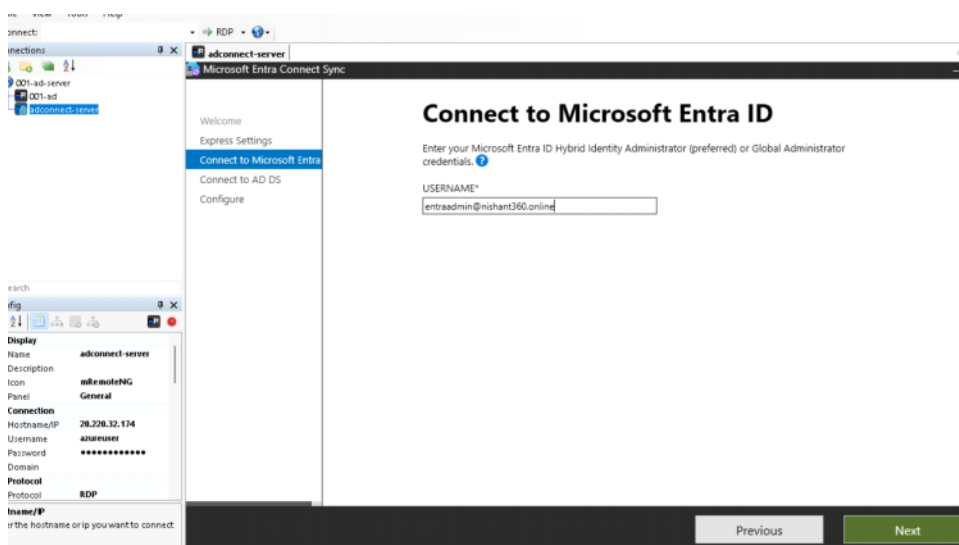
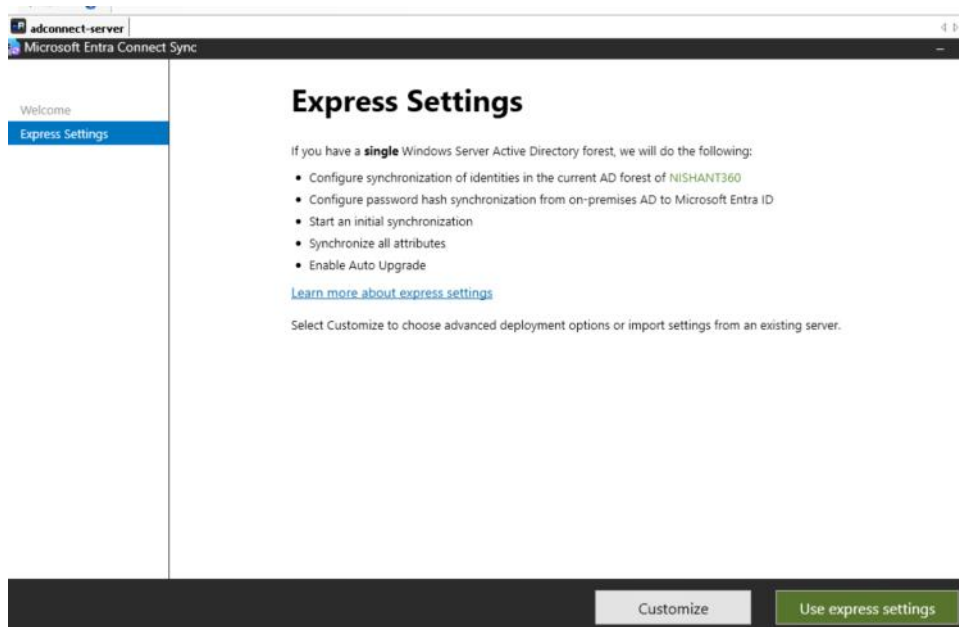
Now user is created
entraadmin@nishant360.online

i will be using this user to connect with azure portal Entra ID to it create the bridge between onprem main active directory server to azure enter id , there is similar kind of process for aws cloud , so all the ad server are in syn with respective server so that there is no need to create user it get sync automatically.

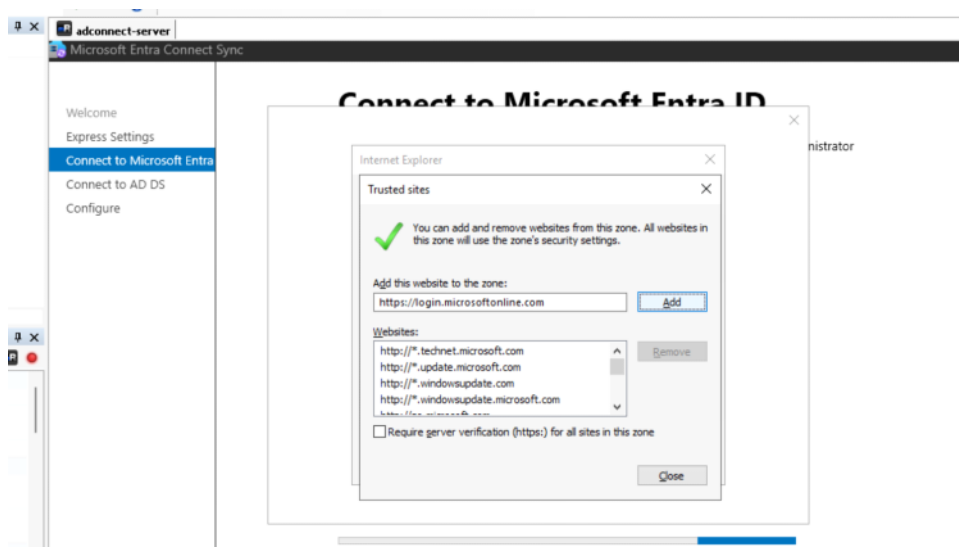
login with that new user private winodw in azure portal rest the password and setup mfa and then login and reset the password

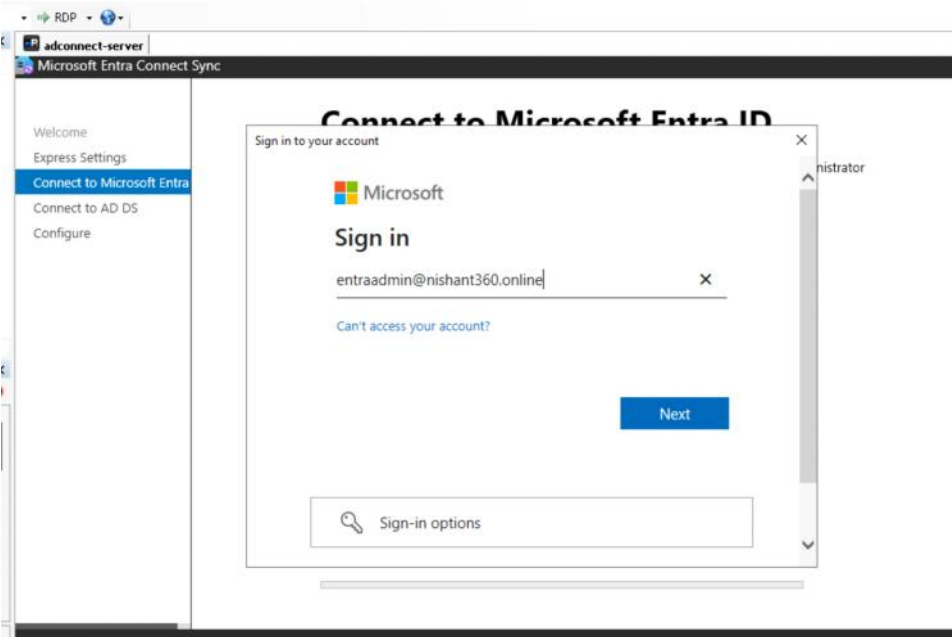
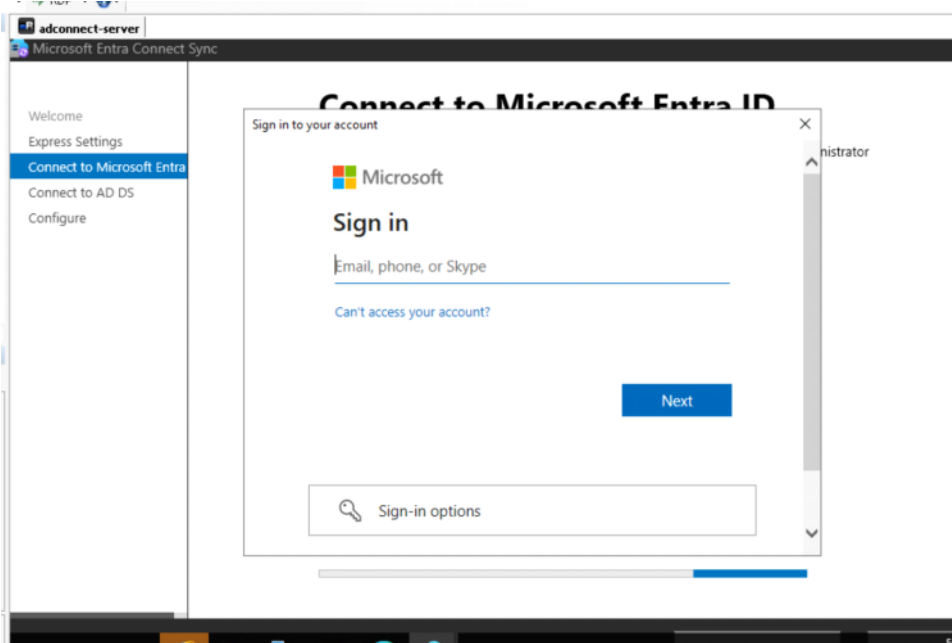
then start setting up ad sync

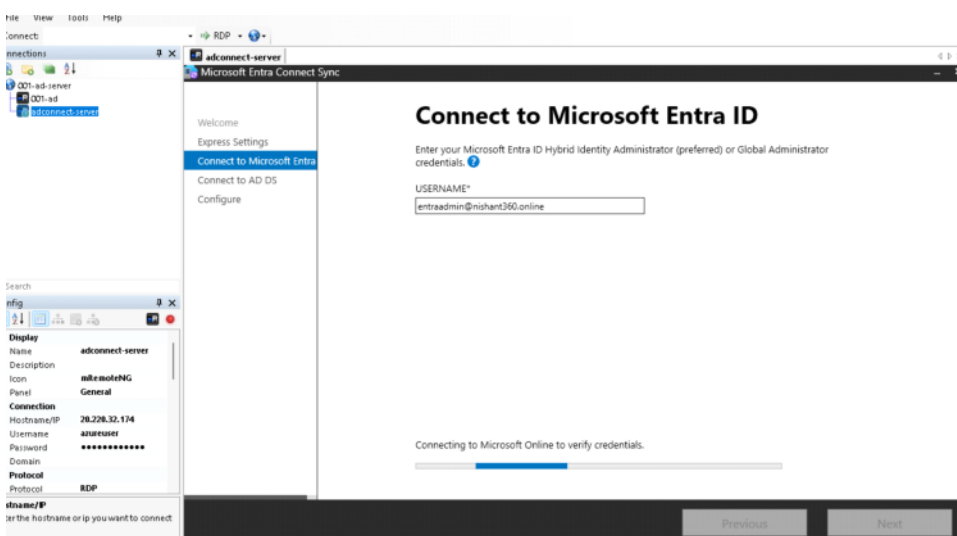
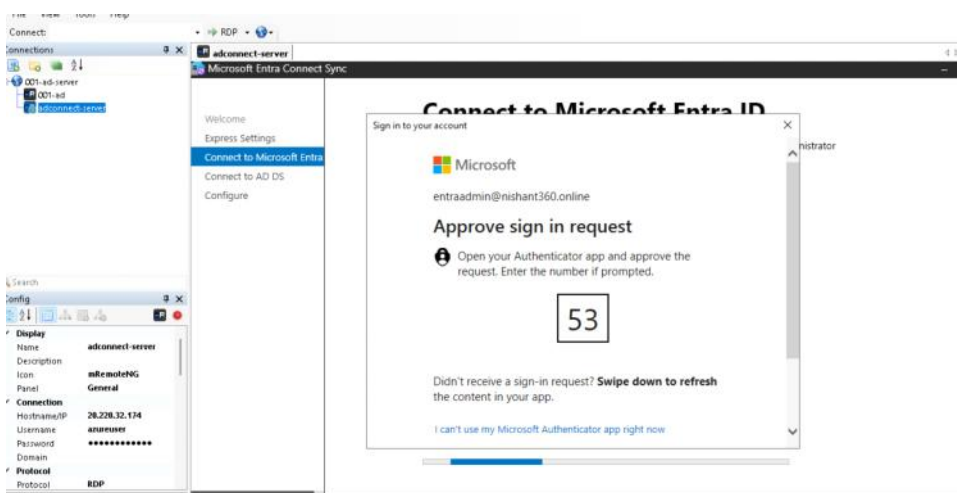
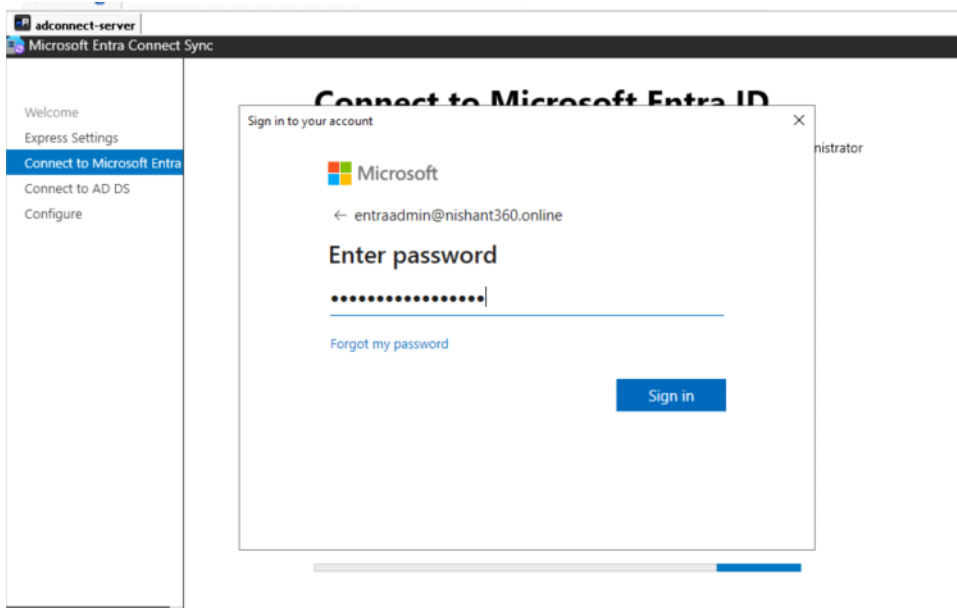




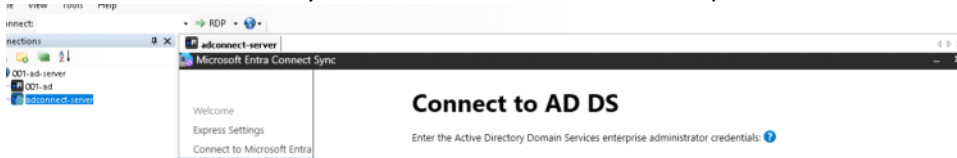
Click on next and keep adding the the trusted site, it will give the popup

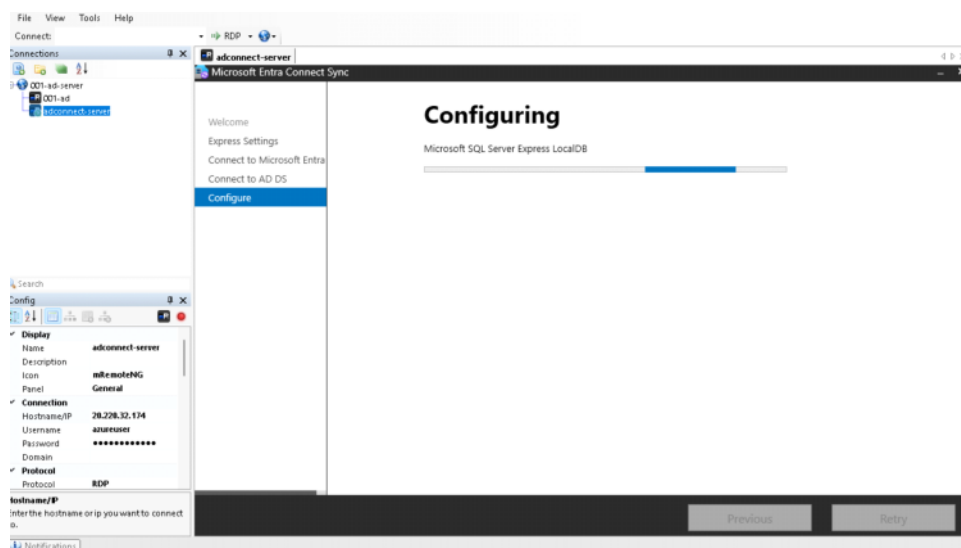
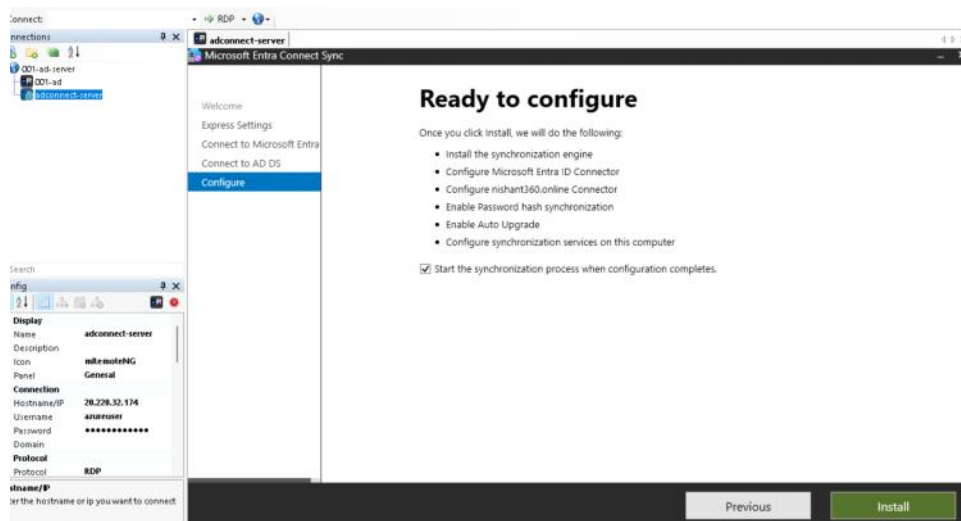
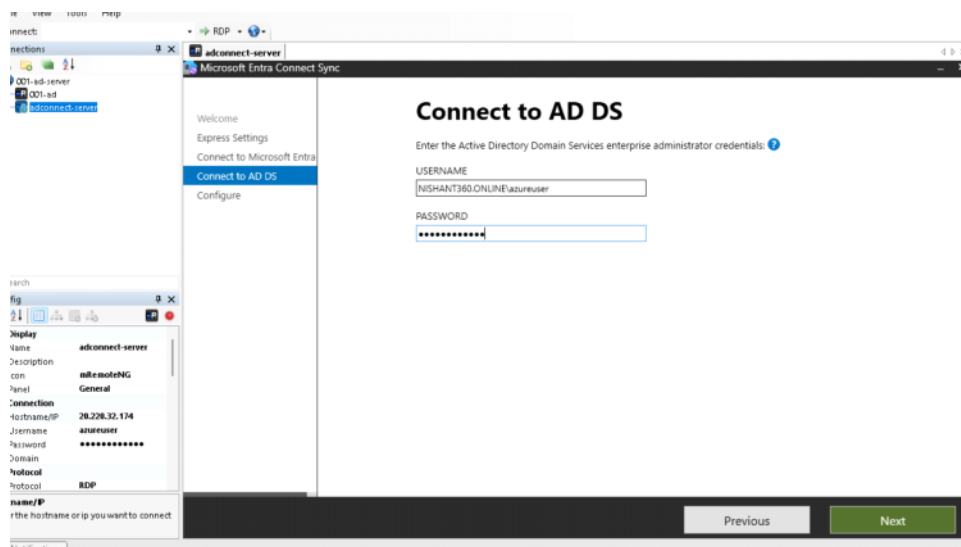


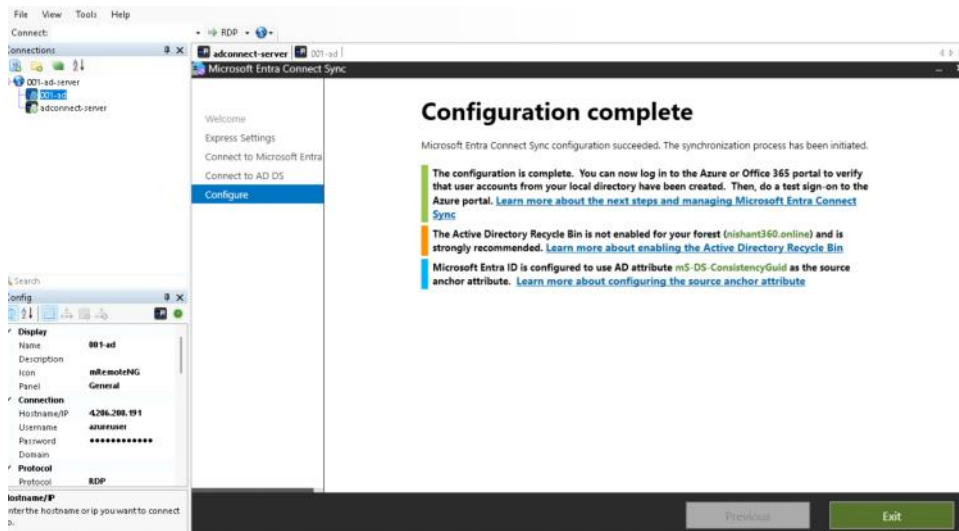




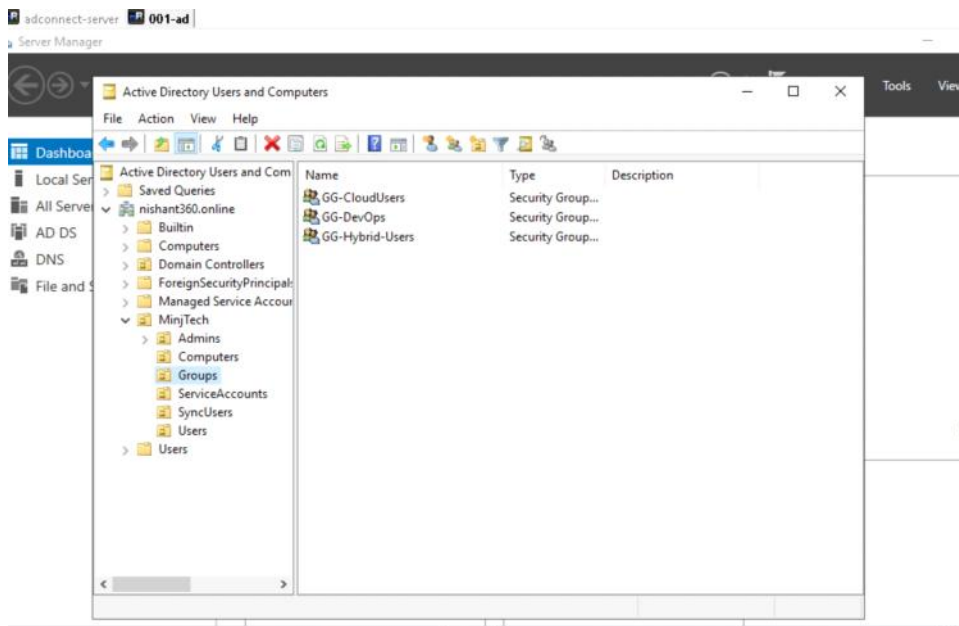
Put below active directory server credential username and password



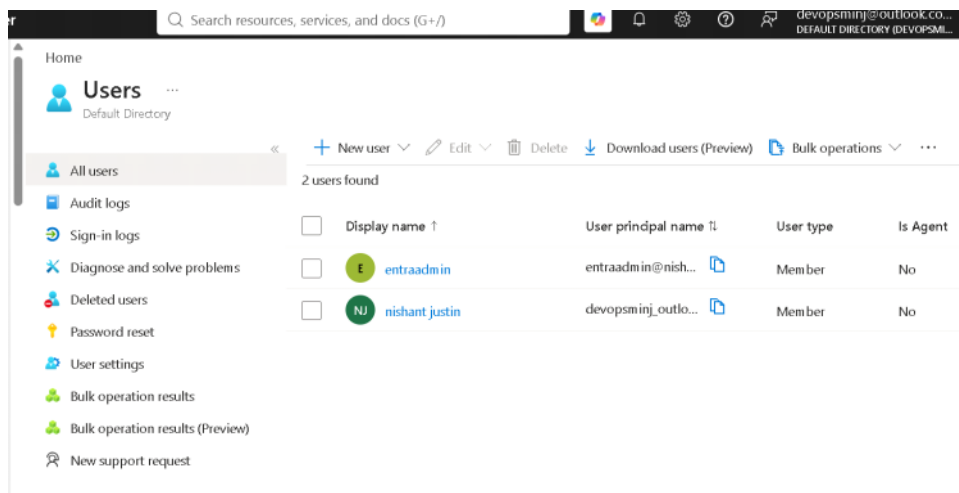




Once sync is done below active directory server group and user automatically sync with the enter id ad server user and group :

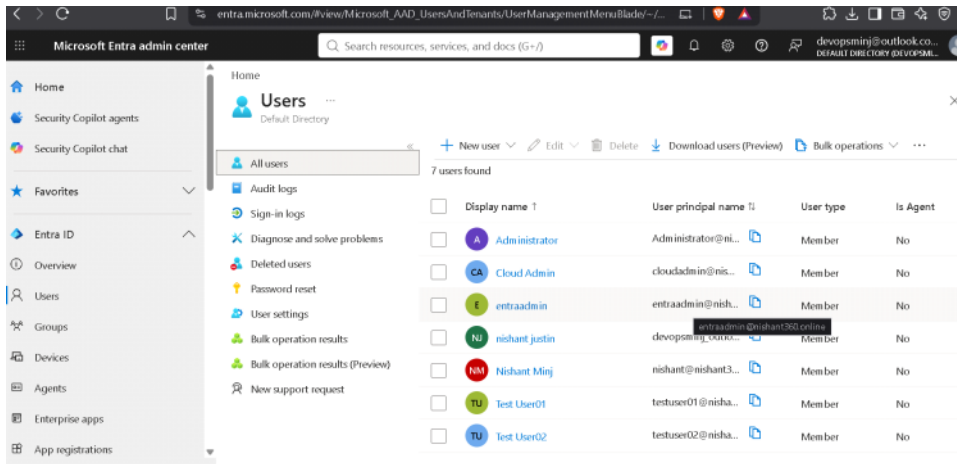


As of now there is only 2 user in enterid



post sync user and group below:

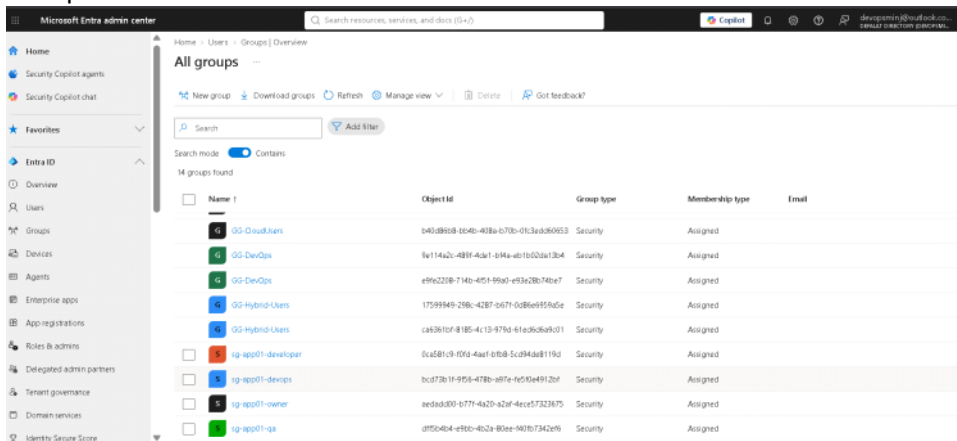
users:



The screenshot shows the Microsoft Entra admin center interface. The left sidebar contains navigation links for Home, Security Copilot agents, Security Copilot chat, Favorites, Entra ID, Overview, Users, Groups, Devices, Agents, Enterprise apps, and App registrations. The main content area is titled 'Users' and shows a list of 7 users found. The table below represents the data shown in the screenshot:

	Display name	User principal name	User type	Is Agent
☐	Administrator	Administrator@ni...	Member	No
☐	Cloud Admin	cloudadmin@nis...	Member	No
☐	entraadmin	entraadmin@nish...	Member	No
☐	nishant.justin	devopsminj@outlook...	Member	No
☐	Nishant Minj	nishant@nishant3...	Member	No
☐	Test User01	testuser01@nisha...	Member	No
☐	Test User02	testuser02@nisha...	Member	No

Group :



The screenshot shows the Microsoft Entra admin center interface for the Groups page. The left sidebar is the same as the previous screenshot. The main content area is titled 'All groups' and shows a list of 14 groups found. The table below represents the data shown in the screenshot:

	Name	Object ID	Group type	Membership type	Email
☐	GG-CloudAdmin	04038108-b04b-403b-b70b-013ed80053	Security	Assigned	
☐	GG-DevOps	1e1142c1-481f-43a1-b14a-eb1002da13b4	Security	Assigned	
☐	GG-DevOps	e96c220b-714d-45d1-89a0-e53a2b74be7	Security	Assigned	
☐	GG-Hybrid-Users	17599949-296c-4287-0671-0086a955a05e	Security	Assigned	
☐	GG-Hybrid-Users	ca93610f-8185-4c13-979d-61ed3c8a9d11	Security	Assigned	
☐	ig-app01-developer	0a5d81cf-f05d-4aa7-b0b8-5ca94da8119d	Security	Assigned	
☐	ig-app01-devops	bcd73b1f-9f5b-478b-a87e-f95f044912af	Security	Assigned	
☐	ig-app01-owner	ae5ad300-b771-4a20-a2af-4ecce57323875	Security	Assigned	
☐	ig-app01-qa	dff50404-e900-4b02-b0ee-907b7342ef6	Security	Assigned	